CĐTN2

LẬP TRÌNH ỨNG DỤNG VỚI FLUTTER

Nguyễn Phồn Lữa, AI

Link mã nguồn: https://github.com/luanp-eaut/cdtn2

Mục lục

Chương 1: Giới thiệu tổng quan về Flutter 5
1.1. Giới thiệu tổng quan về Flutter.....5
1.2. Cấu trúc dự án Flutter.....6
1.3. Các thành phần của một ứng dụng Flutter 8
1.4. Bài tập thực hành 10
1.5. Tổng kết.....11
Chương 2: Ngôn ngữ lập trình Dart..... 13
2.1. Giới thiệu 13
2.2. Tổng quan về Dart 13
2.3. Cú pháp cơ bản 14
2.4. Lập trình hướng đối tượng (OOP).....16
2.5. Null Safety (An toàn null) 19
2.6. Xử lý bất đồng bộ (Asynchronous) 20
2.7. Các tính năng nâng cao 21
2.8. Quản lý gói thư viện (Package Management).....22
2.9. Bài tập thực hành 23
2.10. Tổng kết.....23
Chương 3: Widget trong Flutter..... 24
3.1. Giới thiệu chung 24
3.2. Khái niệm và Đặc điểm của Widget.....24
3.3. Cây Widget (Widget Tree).....25
3.4. Phân loại Widget: Stateless và Stateful 26
3.5. Vòng đời của Stateful Widget (Lifecycle) 29
3.6. Các Widget Layout phổ biến.....29
3.7. Best Practices 31
3.8. Bài tập thực hành 32
3.9. Tổng kết.....33
Chương 4: Quản lý trạng thái (State Management)..... 34
4.1. Giới thiệu 34
4.2. Khái niệm cơ bản về State 34
4.3. Vấn đề với setState và Prop Drilling.....35
4.4. Giải pháp: Tách biệt Logic và UI 36
4.5. Các thư viện quản lý State phổ biến.....36

4.6. Triển khai chi tiết với Provider 37
4.7. Tổng quan về kiến trúc BLoC 39
4.8. So sánh và Lựa chọn giải pháp 40
4.9. Best Practices 40
4.10. Các lỗi thường gặp (Pitfalls) 41
4.11. Bài tập thực hành 42
4.12. Tổng kết 42
Chương 5: Layout & Responsive UI 44
5.1. Giới thiệu 44
5.2. Triết lý Layout của Flutter 44
5.3. Các Widget Layout Cơ Bản 45
5.4. Xây dựng Responsive UI 48
5.5. Scrollable Layouts 50
5.6. Xử lý lỗi Layout thường gặp 51
5.7. Best Practices 52
5.8. Bài tập thực hành 52
5.9. Tổng kết 53
Chương 6: Navigation & Routing 54
6.1. Giới thiệu 54
6.2. Cơ chế Navigation Stack 54
6.3. Điều hướng cơ bản (Imperative Navigation) 54
6.4. Truyền dữ liệu giữa các màn hình 55
6.5. Named Routes (Định tuyến có tên) 57
6.6. Các phương thức điều hướng nâng cao 59
6.7. Xu hướng hiện đại: go_router package 60
6.8. Bài tập thực hành 60
6.9. Tổng kết 61
Chương 7: Style & Theme 63
7.1. Giới thiệu 63
7.2. Khái niệm Style và Theme 63
7.3. Styling cơ bản (Inline Style) 64
7.4. Theme toàn cục với ThemeData 65
7.5. Truy cập dữ liệu Theme trong Widget 67
7.6. Hỗ trợ Dark Mode và Light Mode 68
7.7. Tùy chỉnh Font chữ và Assets 70

7.8. Material Design vs Cupertino 71
7.9. Best Practices 72
7.10. Bài tập thực hành 72
7.11. Tổng kết 73
Chương 8: Quản lý Assets 74
8.1. Giới thiệu 74
8.2. Khái niệm về Assets 74
8.3. Cấu hình Assets trong pubspec.yaml 75
8.4. Làm việc với Hình ảnh (Image Assets) 76
8.5. Resolution Awareness (Đa độ phân giải) 77
8.6. Sử dụng AssetBundle cho dữ liệu phi hình ảnh 78
8.7. Tích hợp Font chữ tùy chỉnh (Custom Fonts) 79
8.8. Sử dụng Assets từ Package bên thứ 3 80
8.9. Tối ưu hóa và Best Practices 81
8.10. Các lỗi thường gặp và cách khắc phục 82
8.11. Bài tập thực hành 83
8.12. Tổng kết 84
Chương 9: Form & Gesture 85
9.1. Giới thiệu 85
9.2. Xử lý Nhập liệu (Input Handling) 85
9.3. Xử lý Cử chỉ (Gesture Handling) 88
9.4. Khả năng tiếp cận (Accessibility) 91
9.5. Các lỗi thường gặp (Pitfalls) 91
9.6. Best Practices 92
9.7. Bài tập thực hành 92
9.8. Tổng kết 93
Chương 10: Animation 94
10.1. Giới thiệu 94
10.2. Phân loại Animation trong Flutter 94
10.3. Implicit Animation 95
10.4. Explicit Animation - Cơ chế cốt lõi 96
10.5. Điều khiển Animation 100
10.6. Các loại Animation đặc biệt 100
10.7. Hiệu suất và Best Practices 101
10.8. Các lỗi thường gặp (Pitfalls) 102

10.9. Bài tập thực hành 102
10.10. Tổng kết 103
Chương 11: Lưu trữ dữ liệu cục bộ 105
11.1. Giới thiệu 105
11.2. Tổng quan các phương thức lưu trữ 105
11.3. Shared Preferences 106
11.4. Cơ sở dữ liệu SQLite với sqflite 108
11.5. Lưu trữ tệp tin (File System) 111
11.6. Bảo mật dữ liệu với Secure Storage 112
11.7. So sánh và lựa chọn phương thức 113
11.8. Best Practices 114
11.9. Bài tập thực hành 115
11.10. Tổng kết 115
Chương 12: Packages và Plugin 117
12.1. Giới thiệu 117
12.2. Tại sao cần sử dụng Packages & Plugins? 117
12.3. Phân biệt Package và Plugin 118
12.4. Kho lưu trữ pub.dev 119
12.5. Cài đặt và Quản lý phiên bản 120
12.6. Sử dụng Package trong Code 121
12.7. Cơ chế hoạt động của Plugin (Platform Channels) 122
12.8. Các vấn đề thường gặp và lưu ý 122
12.9. Tạo Package/Plugin tuỳ biến 124
12.10. Best Practices 124
12.11. Bài tập thực hành 125
12.12. Tổng kết 125

Chương 1: Giới thiệu tổng quan về Flutter

1.1. Giới thiệu tổng quan về Flutter

1.1.1. Flutter là gì?

Flutter là một bộ công cụ giao diện người dùng (UI Toolkit) mã nguồn mở do Google phát triển. Khác với các framework truyền thống chỉ cung cấp thư viện liên kết đến các thành phần native, Flutter cung cấp toàn bộ công cụ cần thiết để vẽ nên giao diện ứng dụng.

Mục tiêu chính của Flutter là cho phép lập trình viên xây dựng các ứng dụng đẹp, nhanh chóng và có hiệu năng cao cho nhiều nền tảng khác nhau (Mobile, Web, Desktop) từ **một mã nguồn duy nhất** (Single Codebase). Ngôn ngữ lập trình được sử dụng trong Flutter là **Dart**, cũng do Google phát triển, được tối ưu hóa đặc biệt cho việc xây dựng giao diện người dùng.

1.1.2. Kiến trúc ba lớp của Flutter

Để hiểu cách Flutter hoạt động, chúng ta cần nhìn vào kiến trúc ba lớp cơ bản của nó:

- **Framework (Lớp khung):** Được viết bằng ngôn ngữ Dart. Đây là nơi lập trình viên tương tác nhiều nhất. Nó cung cấp các thành phần giao diện cơ bản (Widget), các nguyên mẫu thiết kế (Material Design, Cupertino) và cấu trúc ứng dụng.
- **Engine (Lớp động cơ):** Được viết chủ yếu bằng C++. Đây là "trái tim" của Flutter, chịu trách nhiệm xử lý đồ họa (thông qua thư viện Skia hoặc Impeller), xử lý nhập liệu, và truy cập phần cứng. Engine biên dịch mã Dart thành mã máy để thực thi.
- **Embedder (Lớp nhúng):** Được viết bằng ngôn ngữ native của từng nền tảng (Java/Kotlin cho Android, Objective-C/Swift cho iOS, v.v.). Lớp này đóng vai trò cầu nối, giúp Flutter có thể chạy trên hệ điều hành cụ thể và truy cập các dịch vụ hệ thống.

1.1.3. Ưu điểm nổi bật

Flutter đã trở thành một trong những framework phát triển ứng dụng di động phổ biến nhất nhờ những ưu điểm sau:

- **Hot Reload:** Tính năng cho phép xem lại các thay đổi trong code gần như tức thì mà không cần khởi động lại ứng dụng. Điều này giúp giữ nguyên trạng thái (state) hiện tại của ứng dụng, tăng tốc độ phát triển và gỡ lỗi đáng kể.
- **Hiệu năng cao:** Flutter không sử dụng cầu nối JavaScript (như một số framework khác) mà biên dịch trực tiếp ra mã máy native (ARM code). combined với cơ chế tự vẽ giao diện (self-rendering), Flutter đạt được tốc độ khung hình mượt mà (60fps hoặc 120fps).
- **Giao diện đồng nhất:** Vì Flutter tự vẽ các pixel trên màn hình thay vì sử dụng các widget OEM của hệ điều hành, ứng dụng sẽ hiển thị giống nhau trên mọi thiết bị, giảm thiểu rủi ro về sai lệch giao diện khi hệ điều hành cập nhật.
- **Hệ sinh thái phong phú:** Cộng đồng phát triển lớn với kho thư viện pub.dev cung cấp hàng ngàn package hỗ trợ mọi chức năng từ kết nối mạng, cơ sở dữ liệu đến xử lý trạng thái.

1.2. Cấu trúc dự án Flutter

1.2.1. Khởi tạo dự án

Để bắt đầu một dự án mới, Flutter cung cấp câu lệnh tiêu chuẩn trong terminal:

```
flutter create ten_du_an
```

Câu lệnh này sẽ tạo ra một cây thư mục hoàn chỉnh, bao gồm cả các cấu hình cần thiết cho Android, iOS, Web và các nền tảng khác. Tên dự án nên được viết thường và sử dụng dấu gạch dưới (snake_case).

1.2.2. Các thư mục và file quan trọng

Một dự án Flutter sau khi tạo sẽ có cấu trúc như sau:

- **lib/**: Đây là thư mục quan trọng nhất, chứa toàn bộ mã nguồn Dart của ứng dụng. File mặc định main.dart nằm trong này là điểm nhập (entry point) của ứng dụng.
- **pubspec.yaml**: File cấu hình dự án. Nó quản lý thông tin cơ bản (tên, phiên bản), môi trường phát triển, các thư viện phụ thuộc (dependencies) và tài nguyên (assets như hình ảnh, font chữ).
- **android/ và ios/**: Chứa mã nguồn native và cấu hình cụ thể cho từng nền tảng. Lập trình viên Flutter ít khi phải can thiệp vào đây trừ khi cần cấu hình sâu (ví dụ: xin quyền truy cập, thay đổi icon ứng dụng).
- **test/**: Chứa các file kiểm thử (unit test, widget test).
- **build/**: Thư mục chứa các file sinh ra trong quá trình build. Thư mục này thường được thêm vào `.gitignore` vì không cần thiết phải lưu trữ trên hệ thống quản lý phiên bản.

1.2.3. Quản lý phụ thuộc với pubspec.yaml

File pubspec.yaml sử dụng cú pháp YAML, rất nhạy cảm với việc thụt đầu dòng. Một số phần chính cần lưu ý:

- **dependencies**: Khai báo các thư viện bên ngoài. Ví dụ, để sử dụng gói http, chúng ta thêm vào như sau:

```
dependencies:
  flutter:
    sdk: flutter
  http: ^0.13.5
```

Sau khi chỉnh sửa, cần chạy lệnh flutter pub get để tải thư viện về.

- **flutter assets**: Để sử dụng hình ảnh cục bộ, chúng ta phải khai báo đường dẫn trong này thì ứng dụng mới nhận diện được.

1.2.4. Điểm nhập (Entry Point)

Mọi ứng dụng Flutter đều bắt đầu từ hàm main() trong file lib/main.dart.

```
void main() {
  runApp(const MyApp());
}
```

Hàm runApp() có nhiệm vụ gắn widget gốc (trong ví dụ là MyApp) vào màn hình thiết bị. Khác với Android native nơi chúng ta cần khai báo Activity trong Manifest, Flutter chỉ cần gọi runApp() để khởi động giao diện.

1.3. Các thành phần của một ứng dụng Flutter

1.3.1. Tư duy "Mọi thứ là Widget"

Khái niệm quan trọng nhất trong Flutter là **Widget**. Trong Flutter, mọi thành phần của giao diện đều là Widget: từ nút bấm, văn bản, hình ảnh, cho đến các khoảng cách (padding), bố cục (layout) và thậm chí là toàn bộ ứng dụng.

Widget là các khối xây dựng bất biến (immutable). Khi cần thay đổi giao diện, Flutter sẽ tạo ra một cây widget mới thay vì sửa đổi widget cũ. Tư duy này giúp tối ưu hóa việc vẽ lại giao diện.

1.3.2. Cây Widget (Widget Tree)

Flutter tổ chức giao diện theo cấu trúc cây phân cấp, gọi là **Widget Tree**.

- **Widget cha (Parent)**: Chứa và quản lý vị trí của các widget con.
- **Widget con (Child)**: Được hiển thị bên trong widget cha.

Dữ liệu và sự kiện thường truyền từ cha xuống con. Việc hiểu rõ cấu trúc cây giúp lập trình viên dễ dàng debug giao diện và sử dụng công cụ **Flutter Inspector** để xem xét cấu trúc ứng dụng đang chạy.

Ví dụ về cấu trúc cây:

```
Scaffold (Cha)
  ├── AppBar (Con 1)
  └── Body (Con 2)
        └── Column (Cha)
              ├── Text (Con)
              └── ElevatedButton (Con)
```

1.3.3. Phân loại Widget: Stateless và Stateful

Có hai loại widget chính mà lập trình viên cần phân biệt rõ:

- **StatelessWidget (Widget không trạng thái):**
  - Dùng cho các thành phần giao diện không thay đổi sau khi được xây dựng.
  - Dữ liệu được nhận từ bên ngoài thông qua constructor và không thay đổi trong vòng đời của widget.
  - Ví dụ: Icon, Text tĩnh, Image tĩnh.
  - Đặc điểm: Hiệu năng cao, nhẹ.
- **StatefulWidget (Widget có trạng thái):**
  - Dùng cho các thành phần giao diện có thể thay đổi động trong thời gian chạy (ví dụ: khi người dùng nhập liệu, bấm nút).
  - Đi kèm với một đối tượng State riêng biệt để lưu trữ dữ liệu có thể thay đổi.
  - Khi dữ liệu thay đổi, phương thức setState() được gọi để báo cho Flutter vẽ lại widget.
  - *Ví dụ:* Checkbox, TextField, Animation.

Nguyên tắc: Luôn ưu tiên sử dụng `StatelessWidget` khi có thể để tối ưu hiệu năng ứng dụng. Chỉ dùng `StatefulWidget` khi thực sự cần lưu trữ trạng thái thay đổi.

1.3.4. Các Widget Layout cơ bản

Để xây dựng giao diện, Flutter cung cấp các widget bố cục cơ bản sau:

- **Container**: Một hộp chữ nhật đa năng, có thể trang trí màu nền, viền, padding, margin.
- **Row**: Sắp xếp các widget con theo chiều **ngang** (trái sang phải).
- **Column**: Sắp xếp các widget con theo chiều **dọc** (trên xuống dưới).
- **Stack**: Xếp chồng các widget lên nhau theo trục Z (trước sau).
- **ListView**: Tạo danh sách các widget có thể cuộn khi nội dung vượt quá kích thước màn hình.

Lưu ý quan trọng khi dùng Row và Column: Chúng không thể chứa các widget có kích thước vô hạn (như một ListView khác) trừ khi được bọc trong widget Expanded hoặc SizedBox để giới hạn kích thước.

1.3.5. Hệ thống thiết kế (Material và Cupertino)

Flutter hỗ trợ hai hệ thống thiết kế chính để đảm bảo ứng dụng trông tự nhiên trên từng nền tảng:

- **Material Design:** Tuân theo chuẩn thiết kế của Google (thường dùng cho Android). Các widget thường có tiền tố như Scaffold, AppBar, FloatingActionButton.
- **Cupertino:** Tuân theo chuẩn thiết kế của Apple (thường dùng cho iOS). Các widget có tiền tố Cupertino, ví dụ: `CupertinoPageScaffold`, `CupertinoNavigationBar`.

Lập trình viên có thể chọn sử dụng một trong hai hoặc kết hợp linh hoạt tùy theo yêu cầu sản phẩm.

1.4. Bài tập thực hành

Để củng cố kiến thức, người học hãy thực hiện các bài tập sau:

- **Cài đặt môi trường:** Cài đặt Flutter SDK và kiểm tra bằng lệnh flutter doctor. Đảm bảo không có lỗi nào nghiêm trọng.
- **Tạo dự án:** Tạo một dự án mới tên hello_flutter. Chạy thử trên emulator hoặc thiết bị thật.
- **Khám phá cấu trúc:** Mở file pubspec.yaml, thử thêm một package bất kỳ (ví dụ: google_fonts) và chạy flutter pub get.
- **Thử nghiệm Hot Reload:**
  - Mở lib/main.dart.
  - Tìm widget Text hiển thị dòng chữ mặc định.
  - Thay đổi nội dung văn bản, lưu file và quan sát màn hình emulator thay đổi mà không cần reload lại app.
- **Thử nghiệm Layout:** Sử dụng Column và Row để sắp xếp 3 nút bấm theo ý thích trong màn hình chính.

1.5. Tổng kết

Chương 1 đã cung cấp cái nhìn tổng quan về hệ sinh thái Flutter. Chúng ta đã tìm hiểu rằng Flutter không chỉ là một framework mà là một công cụ render giao diện hoàn chỉnh với kiến trúc ba lớp mạnh mẽ.

Những điểm cốt lõi cần ghi nhớ:

- **Single Codebase:** Viết một lần, chạy nhiều nơi.
- **Widget Tree:** Giao diện được xây dựng từ cây các widget lồng nhau.
- **State Management:** Phân biệt rõ khi nào dùng Stateless (tĩnh) và Stateful (động).
- **Cấu trúc dự án:** Làm chủ file pubspec.yaml và thư mục lib/.

Hiểu vững những khái niệm nền tảng này là bước đệm quan trọng để tiếp cận các chủ đề phức tạp hơn như điều hướng (navigation), quản lý trạng thái nâng cao (state management) và kết nối API trong các chương sau.

Chương 2: Ngôn ngữ lập trình Dart

2.1. Giới thiệu

Để trở thành một lập trình viên Flutter thành thạo, việc nắm vững ngôn ngữ lập trình Dart là yêu cầu bắt buộc. Flutter không phải là một ngôn ngữ lập trình riêng biệt mà là một bộ công cụ (framework) được xây dựng hoàn toàn bằng Dart. Do đó, mọi Widget, mọi logic xử lý dữ liệu, và mọi cấu trúc trong Flutter đều tuân theo quy tắc của Dart.

Chương này sẽ cung cấp cho chúng ta nền tảng vững chắc về Dart, từ cú pháp cơ bản đến các tính năng nâng cao như xử lý bất đồng bộ và Null Safety. Hiểu rõ chương này sẽ giúp chúng ta đọc hiểu code Flutter dễ dàng hơn, gỡ lỗi (debug) hiệu quả hơn và viết code tối ưu hơn.

2.2. Tổng quan về Dart

2.2.1. Lịch sử và Mục đích

Dart là ngôn ngữ lập trình do Google phát triển và công bố vào năm 2011. Ban đầu, Dart được thiết kế để thay thế JavaScript trong trình duyệt web, nhưng sau đó đã tìm thấy "đất diễn" thực sự khi trở thành ngôn ngữ cốt lõi của Flutter vào năm 2017.

Dart được thiết kế tối ưu cho việc xây dựng giao diện người dùng (UI). Nó sở hữu những đặc điểm kỹ thuật quan trọng giúp Flutter hoạt động hiệu quả:

- **JIT (Just-In-Time):** Cho phép biên dịch mã nguồn ngay trong quá trình phát triển. Đây là nền tảng cho tính năng **Hot Reload** nổi tiếng của Flutter, giúp lập trình viên thấy thay đổi code gần như tức thì.
- **AOT (Ahead-Of-Time):** Biên dịch mã nguồn thành mã máy (native code) trước khi chạy. Điều này giúp ứng dụng Flutter khi

xuất bản (release) có hiệu năng cao, khởi động nhanh và mượt mà.

- **Garbage Collection:** Tự động quản lý bộ nhớ, giúp lập trình viên không cần lo lắng về việc cấp phát và giải phóng bộ nhớ thủ công.

2.2.2. Môi trường phát triển

Chúng ta có thể viết và chạy code Dart bằng nhiều cách:

- **Dart SDK:** Cài đặt trực tiếp trên máy tính và chạy qua command line.
- **DartPad:** Công cụ trực tuyến tại dartpad.dev, cho phép viết và chạy code Dart ngay trên trình duyệt mà không cần cài đặt. Đây là công cụ tuyệt vời để thử nghiệm các đoạn code ngắn.
- **Flutter SDK:** Khi cài Flutter, chúng ta đã có sẵn Dart SDK đi kèm.

2.3. Cú pháp cơ bản

2.3.1. Điểm bắt đầu (Entry Point)

Mọi chương trình Dart đều bắt đầu thực thi từ hàm main(). Nếu không có hàm này, chương trình sẽ không thể chạy.

```
void main() {
  print('Xin chào Dart!');
}
```

Hàm print() được dùng để in thông tin ra console. Một điểm đặc biệt của Dart là **String Interpolation**, cho phép chèn biến trực tiếp vào chuỗi ký tự:

```
var name = 'Sinh viên';
print('Chào $name'); // Kết quả: Chào Sinh viên
print('Năm sau chúng ta ${2023 + 1} tuổi');
// Kết quả: Năm sau chúng ta 2024 tuổi
```

2.3.2. Biến và Kiểu dữ liệu

Dart là ngôn ngữ có kiểu dữ liệu mạnh (strongly typed), nhưng hỗ trợ suy luận kiểu (type inference).

- **var:** Dùng khi chúng ta muốn Dart tự suy luận kiểu dữ liệu dựa trên giá trị gán ban đầu.
- **Kiểu cụ thể:** Khai báo rõ ràng int, String, bool, double.
- **dynamic:** Biến có thể thay đổi kiểu dữ liệu trong quá trình chạy (hạn chế dùng vì dễ gây lỗi).

```
var age = 20;        // Dart hiểu là int
String name = 'A';   // Khai báo rõ kiểu
dynamic data = 10;   // Có thể đổi thành chuỗi sau
này
data = 'Mười';       // Hợp lệ nhưng không khuyến
khích
```

Hằng số (Constants): Trong Flutter, việc sử dụng hằng số rất quan trọng để tối ưu hiệu năng.

- **final:** Giá trị chỉ được gán một lần tại thời điểm chạy (runtime).
- **const:** Giá trị cố định ngay từ khi biên dịch (compile-time).

```
final currentTime = DateTime.now(); // Giá trị
xác định khi chạy
const pi = 3.14;                    // Giá trị cố
định cứng
const appName = 'My App';           // Nên dùng
const cho Widget không đổi
```

Các kiểu dữ liệu collections:

- **List:** Danh sách có thứ tự.
- **Map:** Collection dạng khóa-giá trị (key-value).
- **Set:** Collection chứa các giá trị duy nhất.

```
List<int> numbers = [1, 2, 3];
Map<String, int> scores = {'Toán': 9, 'Lý': 8};
```

2.3.3. Hàm (Functions)

Hàm trong Dart là đối tượng (first-class citizen), có thể truyền hàm như một biến. Tuy nhiên, điểm đáng chú ý nhất là cách xử lý **tham số**.

- **Tham số bắt buộc (Positional):** Truyền theo thứ tự.
- **Tham số tùy chọn theo vị trí []:** Có thể bỏ qua.
- **Tham số đặt tên {} (Named Parameters):** Rất quan trọng trong Flutter.

```
// Hàm với tham số đặt tên
void createUser({required String name, int age =
18}) {
  print('$name - $age tuổi');
}
// Gọi hàm
createUser(name: 'Nguyen Van A'); // Không cần
truyền age vì có giá trị mặc định
createUser(name: 'Tran Thi B', age: 20);
```

> ***Lưu ý:*** *Từ khóa required (trong Null Safety) bắt buộc người gọi hàm phải truyền giá trị cho tham số name, dù nó là tham số đặt tên. Hầu hết các Widget trong Flutter (như Container, Text, Column) đều sử dụng cơ chế tham số đặt tên này.*

2.4. Lập trình hướng đối tượng (OOP)

Dart là ngôn ngữ thuần hướng đối tượng. Mọi thứ đều là đối tượng, kể cả các kiểu nguyên thủy như số hay boolean.

2.4.1. Class và Constructor

Class là khuôn mẫu để tạo đối tượng. Constructor là hàm đặc biệt dùng để khởi tạo đối tượng.

```
class Person {
  String name;
  int age;
```

```
  // Constructor truyền thống
  Person(this.name, this.age);

  // Constructor đặt tên (Named Constructor)
  Person.guest() : name = 'Guest', age = 0;

  void introduce() {
    print('Tôi là $name');
  }
}
```

Trong Flutter, chúng ta sẽ gặp rất nhiều constructor. Ví dụ: Text('Hello') là gọi constructor mặc định, còn Text.rich(...) là gọi constructor đặt tên.

2.4.2. Kế thừa và Đa hình

Dart hỗ trợ **đơn kế thừa** (một class chỉ có thể kế thừa một class cha) sử dụng từ khóa extends.

```
class Employee extends Person {
  String company;

  Employee(String name, int age, this.company) :
super(name, age);

  @override
  void introduce() {
    super.introduce(); // Gọi hàm của class cha
    print('Làm việc tại $company');
  }
}
```

Trong Flutter, khi tạo một màn hình, chúng ta thường viết: class MyHomePage extends StatefulWidget. Đây chính là áp dụng kế thừa.

2.4.3. Abstract Class và Interface

- **Abstract Class:** Không thể khởi tạo đối tượng trực tiếp, thường dùng để định nghĩa chuẩn chung. Có thể chứa phương thức không có thân hàm (abstract method).
- **Interface:** Dart không có từ khóa interface. Thay vào đó, mọi class đều được coi là interface. Để một class tuân theo interface của class khác, ta dùng từ khóa implements.

```
abstract class Shape {
  double area(); // Phương thức trừu tượng
}

class Circle implements Shape {
  double radius;
  Circle(this.radius);

  @override
  double area() => 3.14 * radius * radius;
}
```

2.4.4. Mixins

Vì Dart chỉ hỗ trợ đơn kế thừa, đôi khi chúng ta muốn một class có thêm tính năng của nhiều class khác mà không cần quan hệ "là một" (is-a). Khi đó, ta dùng **Mixin** với từ khóa with.

```
mixin Flyable {
  void fly() => print('Đang bay');
}

class Bird with Flyable {
  // Class Bird giờ có thêm phương thức fly()
}
```

Trong Flutter, các class State thường dùng mixin, ví dụ: `SingleTickerProviderStateMixin` để hỗ trợ animation.

2.5. Null Safety (An toàn null)

Từ phiên bản Dart 2.12, Null Safety được kích hoạt mặc định. Đây là tính năng giúp loại bỏ lỗi NullPointerException (lỗi truy cập vào đối tượng null) ngay từ khi viết code.

2.5.1. Các ký hiệu quan trọng

- `String`: Biến **không thể** chứa giá trị `null`.
- `String?`: Biến **có thể** chứa giá trị `null`.
- `!`: Toán tử khẳng định (Assertion). Chúng ta cam kết với compiler rằng biến này không null tại thời điểm đó.
- `??`: Toán tử If-null. Nếu biến bên trái null, lấy giá trị bên phải.
- `?.`: Toán tử truy cập an toàn. Nếu đối tượng `null`, trả về `null` thay vì báo lỗi.
- `late`: Khai báo biến sẽ được khởi tạo sau, nhưng cam kết sẽ có giá trị trước khi dùng.

2.5.2. Ví dụ thực tế

```
String? name; // Có thể null

// print(name.length); // Lỗi compile: Tên có thể
null

print(name?.length); // Safe call: Nếu name null
thì kết quả là null
print(name?.length ?? 0); // Nếu null thì lấy giá
trị 0

late String description; // Khởi tạo muộn
// description = 'Mô tả'; // Gán giá trị ở đâu đó
sau này
```

Lời khuyên: *Hãy hạn chế dùng ! trừ khi chúng ta chắc chắn 100% biến không null. Việc lạm dụng ! sẽ vô hiệu hóa lợi ích của Null Safety và có thể gây crash ứng dụng.*

2.6. Xử lý bất đồng bộ (Asynchronous)

Ứng dụng di động cần thực hiện nhiều tác vụ tốn thời gian như gọi API, đọc file, truy vấn database. Nếu thực hiện đồng bộ, giao diện sẽ bị đơ (freeze). Dart sử dụng mô hình đơn luồng với vòng lặp sự kiện (Event Loop) để xử lý bất đồng bộ.

2.6.1. Future

Future đại diện cho một kết quả sẽ có trong tương lai (thành công hoặc thất bại).

```
Future<String> fetchData() async {
  // Giả lập chờ 2 giây
  await Future.delayed(Duration(seconds: 2));
  return 'Dữ liệu đã tải xong';
}
```

2.6.2. Async và Await

Để làm việc với Future dễ dàng, Dart cung cấp cú pháp async và await.

- `async`: Đánh dấu hàm là bất đồng bộ.
- `await`: Dừng thực thi hàm đó cho đến khi Future hoàn thành, nhưng không làm treo toàn bộ ứng dụng.

```
void main() async {
  print('Bắt đầu tải...');
  var result = await fetchData(); // Chờ ở đây
  print(result); // In ra sau 2 giây
  print('Kết thúc');
}
```

Trong Flutter, chúng ta sẽ dùng await rất nhiều khi gọi API bằng package http hoặc dio.

2.6.3. Stream

Nếu Future trả về một giá trị duy nhất trong tương lai, thì Stream trả về một chuỗi các giá trị theo thời gian. Stream thường dùng cho các sự kiện liên tục như nhập liệu, WebSocket, hoặc cảm biến.

```
Stream<int> countStream(int to) async* {
  for (int i = 1; i <= to; i++) {
    yield i; // Phát ra giá trị
  }
}

// Listening stream
countStream(5).listen((value) => print(value));
```

Trong Flutter, Widget StreamBuilder được thiết kế để lắng nghe Stream và cập nhật giao diện tự động khi có dữ liệu mới.

2.7. Các tính năng nâng cao

2.7.1. Generics

Generics cho phép tạo các class hoặc hàm làm việc với nhiều kiểu dữ liệu khác nhau mà vẫn đảm bảo an toàn kiểu. Ký hiệu bằng <T>.

```
class Box<T> {
  T content;
  Box(this.content);
}

var stringBox = Box<String>('Hello');
var intBox = Box<int>(123);
```

Flutter sử dụng Generics extensively, ví dụ: `List<Widget>`, `Future<Response<T>>`.

2.7.2. Extensions

Extensions cho phép chúng ta thêm phương thức mới vào các class có sẵn mà không cần kế thừa hay sửa code gốc của class đó.

```
extension StringExtension on String {
  String capitalize() {
    return
'${this[0].toUpperCase()}${substring(1)}';
  }
}

// Sử dụng
print('hello'.capitalize()); // Kết quả: Hello
```

2.8. Quản lý gói thư viện (Package Management)

Dart có một hệ sinh thái thư viện phong phú được lưu trữ tại pub.dev. Để sử dụng thư viện bên thứ ba, chúng ta cần khai báo trong file pubspec.yaml.

Cấu trúc pubspec.yaml:

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0        # Thư viện gọi API
  provider: ^6.0.0    # Thư viện quản lý trạng
thái
```

- Ký hiệu ^: Cho phép cập nhật phiên bản mới nhất tương thích (ví dụ: từ 1.1.0 lên 1.2.0 nhưng không lên 2.0.0).

Các lệnh thường dùng:

- flutter pub get: Tải thư viện về dự án.
- flutter pub upgrade: Nâng cấp các thư viện lên version mới nhất phù hợp.

- flutter pub publish: Đăng thư viện của chúng ta lên pub.dev (dành cho tác giả).

2.9. Bài tập thực hành

- **Quản lý sinh viên:** Viết một chương trình Dart console sử dụng Class để quản lý danh sách sinh viên (Thêm, Xóa, Tìm kiếm). Sử dụng List và Map.
- **Giả lập API:** Viết một hàm async giả lập việc gọi API (dùng Future.delayed), trả về dữ liệu JSON giả định. Xử lý lỗi bằng try/catch.
- **Extension String:** Tạo một extension cho String để kiểm tra xem chuỗi nhập vào có phải là định dạng email hợp lệ không (sử dụng RegExp).
- **Khám phá thư viện:** Truy cập pub.dev, tìm kiếm thư viện google_fonts, đọc tài liệu và cài đặt vào một dự án Flutter thử nghiệm.

2.10. Tổng kết

Dart là chìa khóa để mở cánh cửa Flutter. Trong chương này, chúng ta đã đi qua:

- **Cú pháp nền tảng:** Biến, hàm, và đặc biệt là tham số đặt tên.
- **OOP:** Class, kế thừa, và mixin – nền tảng của hệ thống Widget.
- **Null Safety:** Cơ chế bảo vệ ứng dụng khỏi lỗi null nguy hiểm.
- **Bất đồng bộ:** Future và Stream để xử lý dữ liệu mạng và sự kiện.
- **Hệ sinh thái:** Cách sử dụng thư viện từ pub.dev.

Việc luyện tập viết code Dart thuần túy trước khi nhảy vào Widget sẽ giúp chúng ta tiết kiệm thời gian gỡ lỗi về sau.

Chương 3: Widget trong Flutter

3.1. Giới thiệu chung

Nếu như viên gạch là đơn vị cơ bản để xây dựng nên một ngôi nhà, thì **Widget** chính là đơn vị cơ bản để xây dựng nên giao diện người dùng (UI) trong Flutter. Triết lý thiết kế cốt lõi của Flutter được gói gọn trong câu nói: **"Everything is a Widget"** (Mọi thứ đều là Widget).

Khác với các nền tảng phát triển ứng dụng truyền thống như Android (sử dụng View) hay iOS (sử dụng UIView), nơi các thành phần giao diện và logic thường tách biệt, Flutter thống nhất mọi thứ dưới dạng Widget. Từ những thành phần hiển thị cơ bản như văn bản, nút bấm, hình ảnh, cho đến các thành phần cấu trúc như khoảng cách, căn lề, hay thậm chí là toàn bộ ứng dụng, tất cả đều được mô tả bằng Widget.

Chương này sẽ đi sâu vào bản chất của Widget, cách chúng hoạt động, phân loại và cách sử dụng các Widget layout phổ biến để xây dựng giao diện ứng dụng hoàn chỉnh.

3.2. Khái niệm và Đặc điểm của Widget

3.2.1. Widget là gì?

Trong Flutter, Widget là một lớp (class) mô tả cách một phần của giao diện người dùng nên hiển thị dựa trên trạng thái hiện tại của nó. Điều quan trọng cần hiểu là **Widget không phải là thành phần UI trực tiếp** (như pixel trên màn hình), mà nó là **cấu hình** (configuration) để tạo ra thành phần UI đó.

Khi trạng thái ứng dụng thay đổi, Flutter sẽ tạo lại các Widget bị ảnh hưởng để phản ánh sự thay đổi đó. Quá trình này diễn ra rất nhanh nhờ vào engine hiệu năng cao của Flutter.

3.2.2. Mô hình Declarative (Khai báo)

Flutter sử dụng mô hình lập trình **Declarative** (Khai báo), đối lập với mô hình **Imperative** (Mệnh lệnh) thường thấy trong Android native hay iOS native.

- **Imperative (Mệnh lệnh):** Chúng ta tạo ra một đối tượng UI, sau đó ra lệnh thay đổi thuộc tính của nó khi cần. *Ví dụ:* `button.setText("New Text")`
- **Declarative (Khai báo):** Chúng ta mô tả giao diện sẽ như thế nào ở một trạng thái cụ thể. Khi trạng thái thay đổi, Flutter sẽ tự động xây dựng lại giao diện mới phù hợp với mô tả đó. *Ví dụ:* `Text(isLoading ? "Đang tải..." : "Hoàn tất")`

Cách tiếp cận Declarative giúp code dễ đọc, dễ bảo trì và giảm thiểu các lỗi liên quan đến việc đồng bộ trạng thái giao diện thủ công.

3.3. Cây Widget (Widget Tree)

Giao diện trong Flutter được tổ chức dưới dạng một cây phân cấp, gọi là **Widget Tree**. Một Widget có thể chứa nhiều Widget con (children), và các Widget con này lại có thể chứa các Widget con khác.

3.3.1. Cấu trúc phân cấp

Mối quan hệ trong cây Widget là quan hệ **Cha (Parent) - Con (Child)**.

- **Widget gốc:** Thường là `MaterialApp` (cho Android/Multi-platform) hoặc `CupertinoApp` (cho iOS).
- **Widget trung gian:** Các widget layout như `Scaffold, Column, Row, Container`.
- **Widget lá:** Các widget hiển thị cuối cùng như Text, Icon, Image.

Ví dụ về cấu trúc cây:

```
MaterialApp
└── Scaffold
```

```
├── AppBar (Text: "Trang chủ")
└── Body (Column)
    ├── Text ("Xin chào")
    └── ElevatedButton (Text: "Bấm vào tôi")
```

3.3.2. Nguyên tắc Layout

Khi Flutter thực hiện layout, nó tuân theo một quy tắc vàng: **"Constraints go down, sizes go up, parent sets position"** (Ràng buộc đi xuống, kích thước đi lên, cha quyết định vị trí).

- Widget cha truyền các ràng buộc (constraints) xuống widget con (ví dụ: chiều rộng tối đa là bao nhiêu).
- Widget con quyết định kích thước của nó dựa trên các ràng đó và thông báo lại cho cha.
- Widget cha quyết định vị trí hiển thị của widget con.

Hiểu rõ nguyên tắc này giúp lập trình viên giải quyết các lỗi layout phổ biến như RenderFlex overflowed (Nội dung quá lớn so với khung chứa).

3.4. Phân loại Widget: Stateless và Stateful

Trong Flutter, có hai loại Widget chính mà lập trình viên cần phân biệt rõ ràng: **Stateless Widget** và **Stateful Widget**.

3.4.1. Stateless Widget (Widget không trạng thái)

Stateless Widget là loại widget không lưu trữ trạng thái thay đổi theo thời gian. Một khi nó được xây dựng, các thuộc tính của nó là bất biến (immutable).

- Đặc điểm:
  - Chỉ nhận dữ liệu đầu vào thông qua constructor.
  - Phương thức duy nhất cần implement là build().
  - Hiệu năng cao hơn do không cần quản lý vòng đời phức tạp.
- Khi nào sử dụng?

- Khi giao diện không thay đổi sau khi hiển thị (ví dụ: Icon, Text tĩnh).
- Khi giao diện chỉ phụ thuộc vào thông tin truyền từ widget cha.

Ví dụ:

```
class MyTextWidget extends StatelessWidget {
  final String message;

  const MyTextWidget({Key? key, required
this.message}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Text(message);
  }
}
```

Lưu ý: *Nên sử dụng từ khóa const cho constructor nếu widget không có tham số thay đổi để tối ưu hóa bộ nhớ.*

3.4.2. Stateful Widget (Widget có trạng thái)

Stateful Widget là loại widget có thể thay đổi giao diện trong suốt vòng đời của nó. Nó quản lý một đối tượng State riêng biệt để lưu trữ dữ liệu động.

- Đặc điểm:
  - Gồm 2 class: Class Widget (khai báo cấu hình) và Class State (quản lý dữ liệu).
  - Có thể gọi phương thức setState() để thông báo cho Flutter biết trạng thái đã thay đổi và cần vẽ lại giao diện.
- Khi nào sử dụng?
  - Khi giao diện cần phản hồi lại tương tác người dùng (ví dụ: Checkbox, Form nhập liệu).

- Khi cần thực hiện animation.
- Khi cần lấy dữ liệu động từ API và hiển thị lại.

Ví dụ:

```
class CounterWidget extends StatefulWidget {
  @override
  _CounterWidgetState createState() =>
_CounterWidgetState();
}

class _CounterWidgetState extends
State<CounterWidget> {
  int _count = 0;

  void _increment() {
    setState(() {
      _count++; // Cập nhật trạng thái
    });
  }

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: _increment,
      child: Text('Đã bấm: $_count lần'),
    );
  }
}
```

Giải thích: Khi `_increment` được gọi, `setState` báo hiệu cho Flutter rằng dữ liệu `_count` đã thay đổi. Flutter sẽ gọi lại phương thức `build()` để cập nhật giao diện mới. Nếu thiếu `setState`, biến `_count` vẫn tăng nhưng giao diện sẽ không thay đổi.

3.5. Vòng đời của Stateful Widget (Lifecycle)

Hiểu rõ vòng đời của Stateful Widget giúp lập trình viên quản lý tài nguyên hiệu quả và tránh các lỗi rò rỉ bộ nhớ (memory leak). Các phương thức chính theo thứ tự gọi:

- **createState()**: Tạo đối tượng State liên kết với Widget.
- **initState()**: Được gọi ngay sau khi State được chèn vào cây Widget.
  - *Công dụng:* Khởi tạo biến, đăng ký listener, gọi API lần đầu.
  - *Lưu ý:* Chỉ được gọi **một lần** duy nhất. Không được dùng context ở đây để truy cập InheritedWidget phụ thuộc vào vị trí cây.
- **didChangeDependencies()**: Gọi ngay sau initState() hoặc khi dependencies thay đổi.
- **build()**: Xây dựng giao diện. Phương thức này có thể được gọi **nhiều lần** bất cứ khi nào trạng thái thay đổi.
- **didUpdateWidget()**: Gọi khi Widget cha cập nhật cấu hình và cần rebuild Widget hiện tại.
- **setState()**: Phương thức do lập trình viên gọi để kích thích rebuild.
- **deactivate()**: Gọi khi Widget bị loại khỏi cây Widget (nhưng có thể được chèn lại).
- **dispose()**: Gọi khi Widget bị loại khỏi cây vĩnh viễn.
  - *Công dụng:* Dọn dẹp tài nguyên (hủy stream, dispose controller, remove listener).
  - *Lưu ý:* **Bắt buộc** phải override phương thức này để tránh rò rỉ bộ nhớ.

3.6. Các Widget Layout phổ biến

Để xây dựng giao diện phức tạp, chúng ta cần kết hợp nhiều Widget layout khác nhau. Chúng được chia thành các nhóm chính sau:

3.6.1. Widget chứa một con (Single Child Widgets)

- **Container:** Widget đa năng nhất. Cho phép thiết lập màu nền, padding, margin, border, kích thước cố định.

  Lưu ý: *Container khá "nặng". Nếu chỉ cần padding, hãy dùng widget Padding riêng biệt để tối ưu hiệu năng.*

- **Padding:** Tạo khoảng đệm bên trong cho widget con.
- **Center:** Căn giữa widget con theo cả chiều ngang và dọc.
- **SizedBox:** Dùng để đặt kích thước cố định (width/height) hoặc tạo khoảng trống (spacer). *Ví dụ:* `SizedBox(height: 20)` thường dùng để tạo khoảng cách giữa các dòng.
- **Align:** Căn vị trí widget con trong khung cha (top, bottom, center...).

3.6.2. Widget chứa nhiều con (Multi Child Widgets)

- **Row:** Sắp xếp các widget con theo chiều **ngang**.
- **Column:** Sắp xếp các widget con theo chiều **dọc**.
  - Thuộc tính quan trọng:
    - `mainAxisAlignment`: Căn chỉnh theo trục chính (dọc với Column, ngang với Row).
    - `crossAxisAlignment`: Căn chỉnh theo trục phụ.
    - `children`: Danh sách các widget con.
  - Lỗi thường gặp: RenderFlex overflowed. Xảy ra khi tổng kích thước các con lớn hơn kích thước màn hình.
  - Cách sửa: Bọc Row/Column trong `SingleChildScrollView` hoặc dùng `Expanded/Flexible` cho các widget con.
- **Stack:** Chồng các widget lên nhau theo trục Z (layer).

  - Thường dùng kết hợp với Positioned để định vị tuyệt đối các widget con (ví dụ: Badge thông báo trên icon).

- **ListView:** Danh sách có thể cuộn.
  - ListView: Tạo tất cả widget ngay lập tức (chỉ dùng cho danh sách ngắn).
  - ListView.builder: Tạo widget lazy (chỉ tạo khi cần hiển thị). **Khuyên dùng** cho danh sách dài hoặc dữ liệu từ API.
- **GridView:** Hiển thị danh sách dưới dạng lưới (nhiều cột).

3.6.3. Material vs Cupertino Widgets

Flutter cung cấp hai bộ widget thiết kế để phù hợp với hai nền tảng chính:

- **Material Widgets:** Tuân theo chuẩn Material Design của Google (phổ biến cho Android và Web). Ví dụ: `Scaffold`, `AppBar`, `FloatingActionButton`, `ElevatedButton`.
- **Cupertino Widgets:** Tuân theo chuẩn thiết kế của Apple (iOS). Ví dụ: `CupertinoNavigationBar`, `CupertinoButton`, `CupertinoSwitch`.

Lập trình viên có thể trộn lẫn hai bộ widget này, nhưng để trải nghiệm người dùng tốt nhất, nên nhất quán theo phong cách của nền tảng mục tiêu hoặc sử dụng các widget thích ứng (adaptive widgets).

3.7. Best Practices

Để viết code Flutter sạch, hiệu năng cao và dễ bảo trì, hãy tuân thủ các nguyên tắc sau:

- **Sử dụng const khi có thể:** Khi một Widget không thay đổi thuộc tính, hãy khai báo nó bằng const. Điều này giúp Flutter không cần xây dựng lại đối tượng đó khi widget cha rebuild, tiết kiệm bộ nhớ và tăng tốc độ render.

```
// Tốt
const Text('Hello')
// Chưa tối ưu
Text('Hello')
```

- **Tách nhỏ Widget (Extract Widget):** Tránh viết toàn bộ giao diện trong một phương thức build khổng lồ. Hãy tách các phần giao diện phức tạp thành các Widget riêng biệt (thường là StatelessWidget).
  - *Lợi ích:* Dễ đọc, dễ tái sử dụng, và giúp tối ưu rebuild (chỉ rebuild phần widget con thay đổi).
- **Quản lý State đúng chỗ:** Chỉ sử dụng Stateful Widget khi thực sự cần thiết. Nếu một phần giao diện không cần thay đổi, hãy giữ nó là Stateless. Đưa trạng thái lên mức cao nhất cần thiết (State Lift) để dễ quản lý.
- **Luôn gọi dispose():** Nếu chúng ta sử dụng TextEditingController, AnimationController, hoặc đăng ký StreamSubscription, hãy chắc chắn hủy chúng trong phương thức dispose() để tránh rò rỉ bộ nhớ.
- **Sử dụng Key khi cần thiết:** Khi danh sách các widget con có thể thay đổi thứ tự, bị xóa hoặc thêm mới mà vẫn cần giữ trạng thái (ví dụ: giữ nội dung nhập trong TextField), hãy sử dụng Key (như ValueKey, ObjectKey) để giúp Flutter nhận diện đúng widget.

3.8. Bài tập thực hành

Bài tập 1: Giao diện Login

- Yêu cầu: Xây dựng màn hình đăng nhập đơn giản.
- Thành phần: Logo (Image), 2 ô nhập liệu (TextField - Username, Password), Nút đăng nhập (ElevatedButton).
- Kỹ thuật: Sử dụng Column, Container, Padding, SizedBox.

Bài tập 2: Danh sách sản phẩm

- Yêu cầu: Hiển thị danh sách 20 sản phẩm giả lập.
- Thành phần: Mỗi sản phẩm gồm Ảnh, Tên, Giá.
- Kỹ thuật: Sử dụng ListView.builder, Card, Row.
- Nâng cao: Thêm chức năng bấm vào sản phẩm để tăng số lượng mua (sử dụng Stateful Widget và setState).

Bài tập 3: Tìm hiểu lỗi Layout

- Yêu cầu: Tạo một Column chứa nhiều Text sao cho gây ra lỗi RenderFlex overflowed.
- Nhiệm vụ: Tìm hiểu nguyên nhân và sửa lỗi bằng cách sử dụng SingleChildScrollView hoặc Expanded.

3.9. Tổng kết

Widget là nền tảng của mọi ứng dụng Flutter. Việc nắm vững cách phân loại Widget (Stateless vs Stateful), hiểu rõ vòng đời và thành thạo các Widget layout là kỹ năng bắt buộc để phát triển ứng dụng chuyên nghiệp.

Chương 4: Quản lý trạng thái (State Management)

4.1. Giới thiệu

Trong lập trình ứng dụng di động nói chung và Flutter nói riêng, **trạng thái (State)** là trái tim của sự tương tác. Một ứng dụng tĩnh chỉ hiển thị thông tin sẽ rất nhàm chán. Sức mạnh thực sự của Flutter nằm ở khả năng phản ứng lại các hành động của người dùng, dữ liệu từ máy chủ, hoặc cảm biến thiết bị bằng cách cập nhật giao diện người dùng (UI) một cách linh hoạt.

Công thức cốt lõi để hiểu về Flutter là: $UI = f(State)$

Điều này có nghĩa là Giao diện người dùng (UI) là một hàm số của Trạng thái (State). Khi trạng thái thay đổi, Flutter sẽ tự động vẽ lại (rebuild) các widget liên quan để phản ánh trạng thái mới đó. Chương này sẽ đi sâu vào cách tổ chức, lưu trữ và quản lý các trạng thái đó sao cho hiệu quả, dễ bảo trì và mở rộng.

4.2. Khái niệm cơ bản về State

4.2.1. State là gì?

Hiểu một cách đơn giản, **State** là bất kỳ dữ liệu nào có thể thay đổi trong suốt vòng đời của ứng dụng. Khi dữ liệu này thay đổi, giao diện cần được cập nhật để người dùng thấy được sự thay đổi đó.

Ví dụ về State trong thực tế:

- **Dữ liệu nhập liệu:** Nội dung người dùng gõ vào ô TextField.
- **Trạng thái UI:** Một nút bấm đang ở trạng thái "Đã thích" (icon trái tim đỏ) hay "Chưa thích" (icon trái tim trắng).
- **Dữ liệu nghiệp vụ:** Danh sách sản phẩm trong giỏ hàng, thông tin tài khoản người dùng đăng nhập, kết quả trả về từ API.
- **Cấu hình:** Chế độ sáng/tối (Dark/Light mode), ngôn ngữ hiển thị.

4.2.2. Phân loại State

Không phải trạng thái nào cũng cần được quản lý theo cùng một cách. Trong Flutter, chúng ta phân chia State thành hai loại chính:

- Ephemeral State (Trạng thái nhất thời):
  - Còn gọi là UI State.
  - Chỉ liên quan đến một Widget duy nhất hoặc một màn hình cụ thể.
  - Ví dụ: currentIndex của một BottomNavigationBar, trạng thái isLoading của một nút bấm, hiệu ứng animation đơn giản.
  - **Cách quản lý:** Sử dụng StatefulWidget và phương thức setState().
- App State (Trạng thái ứng dụng):
  - Là dữ liệu dùng chung, có thể được truy cập từ nhiều Widget, nhiều màn hình khác nhau.
  - Ví dụ: Thông tin User (để hiển thị ở trang Profile và gửi kèm khi đặt hàng), Giỏ hàng (hiển thị ở trang chủ và trang thanh toán), Cài đặt ngôn ngữ toàn app.
  - **Cách quản lý:** Cần các giải pháp State Management chuyên biệt (Provider, BLoC, Riverpod...).

4.3. Vấn đề với setState và Prop Drilling

Đối với các ứng dụng nhỏ hoặc trạng thái nhất thời, `setState()` là giải pháp hoàn hảo. Tuy nhiên, khi ứng dụng phát triển lớn hơn, việc phụ thuộc hoàn toàn vào setState sẽ gặp nhiều vấn đề nghiêm trọng.

4.3.1. Hiện tượng Prop Drilling

Khi một trạng thái cần được sử dụng ở nhiều nơi sâu trong cây Widget, chúng ta buộc phải truyền dữ liệu đó từ Widget cha xuống Widget con thông qua các constructor. Nếu có nhiều tầng trung gian

không cần dữ liệu đó nhưng vẫn phải truyền qua, hiện tượng này gọi là **Prop Drilling**.

Hậu quả:

- Code trở nên rối rắm, khó đọc.
- Khó bảo trì: Khi thay đổi cấu trúc dữ liệu, phải sửa ở nhiều nơi.
- Các Widget trung gian bị phụ thuộc không cần thiết vào dữ liệu mà chúng không dùng.

4.3.2. Hiệu năng và Khả năng kiểm thử

- **Hiệu năng:** Gọi setState ở Widget cha có thể kích hoạt việc vẽ lại toàn bộ cây con bên dưới, kể cả những Widget không liên quan đến dữ liệu thay đổi, gây giật lag.
- **Kiểm thử (Testing):** Logic nghiệp vụ nằm chìm trong các Widget UI khiến việc viết Unit Test trở nên khó khăn. Chúng ta không thể test logic mà không cần khởi tạo cả môi trường UI.

4.4. Giải pháp: Tách biệt Logic và UI

Để giải quyết các vấn đề trên, nguyên tắc vàng là **Tách biệt Logic và UI**. Chúng ta cần đưa State ra khỏi cây Widget (Widget Tree) và quản lý nó ở một nơi độc lập.

Lợi ích của việc tách biệt:

- **Code sạch hơn:** Tuân thủ nguyên tắc Clean Architecture.
- **Dễ bảo trì:** Thay đổi logic không ảnh hưởng đến UI và ngược lại.
- **Dễ kiểm thử:** Có thể viết Unit Test cho logic nghiệp vụ độc lập.
- **Tối ưu hiệu năng:** Chỉ những Widget nào thực sự lắng nghe sự thay đổi của State mới được vẽ lại.

4.5. Các thư viện quản lý State phổ biến

Hệ sinh thái Flutter có nhiều giải pháp quản lý trạng thái. Dưới đây là những cái tên phổ biến nhất:

- **Provider:** Giải pháp được Google khuyên dùng cho người mới bắt đầu. Dựa trên InheritedWidget, dễ học, ít code mẫu (boilerplate).
- **Riverpod:** Phiên bản cải tiến của Provider, khắc phục các nhược điểm về phụ thuộc BuildContext, an toàn kiểu dữ liệu hơn (compile-safe).
- **BLoC (Business Logic Component):** Dựa trên Stream và Pattern. Tách biệt hoàn toàn logic và UI. Phù hợp cho dự án lớn, doanh nghiệp.
- **GetX:** Nhẹ, tốc độ cao, tích hợp nhiều tính năng (Navigation, State, Dependency Injection). Tuy nhiên, cách tiếp cận này gây tranh cãi về tính chuẩn hóa lâu dài.
- **MobX:** Dựa trên reactive programming, tự động theo dõi sự thay đổi biến.

4.6. Triển khai chi tiết với Provider

Provider là điểm khởi đầu lý tưởng để hiểu về quản lý trạng thái. Nó hoạt động dựa trên mô hình **Observer Pattern**.

4.6.1. Cơ chế hoạt động

- Tạo một class quản lý dữ liệu kế thừa từ ChangeNotifier.
- Cung cấp (Provide) class đó vào cây Widget ở vị trí cao nhất có thể.
- Các Widget con lắng nghe (Listen) sự thay đổi.
- Khi dữ liệu thay đổi, gọi notifyListeners(), các Widget lắng nghe sẽ tự động cập nhật.

4.6.2. Các bước thực hiện

Bước 1: Tạo Model quản lý State Tạo một class kế thừa ChangeNotifier. Các biến dữ liệu nên là private, cung cấp getter public và phương thức để thay đổi dữ liệu.

```
import 'package:flutter/foundation.dart';
```

```
class CounterModel extends ChangeNotifier {
  int _count = 0;

  // Getter để UI đọc dữ liệu
  int get count => _count;

  // Phương thức thay đổi trạng thái
  void increment() {
    _count++;
    notifyListeners(); // Thông báo cho UI biết
dữ liệu đã thay đổi
  }

  void decrement() {
    _count--;
    notifyListeners();
  }
}
```

Bước 2: Cung cấp State (Provide) Sử dụng `ChangeNotifierProvider` để bao quanh ứng dụng hoặc một nhánh Widget cụ thể.

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterModel(),
      child: MyApp(),
    ),
  );
}
```

Bước 3: Tiêu thụ State (Consume) Có hai cách phổ biến để lấy dữ liệu trong UI:

- **Cách 1: Sử dụng Consumer** Widget này tự động lắng nghe và rebuild khi có thay đổi.

```
Consumer<CounterModel>(
  builder: (context, counter, child) {
    return Text('Số lần bấm: ${counter.count}');
  },
)
```

- **Cách 2: Sử dụng context.watch hoặc context.read**
  - `context.watch<T>()`: Dùng khi cần rebuild Widget khi dữ liệu thay đổi (dùng trong hàm `build`).
  - `context.read<T>()`: Dùng khi chỉ cần lấy dữ liệu để xử lý logic (ví dụ trong hàm `onPressed`), không cần rebuild. ```dart // Trong hàm build final count = context.watch().count;
    `// Trong hàm onPressed onPressed: () => context.read().increment();`

4.7. Tổng quan về kiến trúc BLoC

Nếu Provider phù hợp cho ứng dụng vừa và nhỏ, thì **BLoC** là tiêu chuẩn cho các ứng dụng doanh nghiệp lớn.

4.7.1. Triết lý BLoC

BLoC dựa trên nguyên tắc: **Mọi thứ đều là Stream**.

- **Input:** Các sự kiện (Events) từ UI (ví dụ: `LoginButtonPressed`, `RefreshRequested`).
- **Process:** Bloc nhận Event, xử lý logic nghiệp vụ, gọi API.
- **Output:** Phát ra các trạng thái (States) mới (ví dụ: `LoadingState`, `SuccessState`, `ErrorState`).

4.7.2. Luồng dữ liệu

(1) Người dùng tương tác UI → Gửi **Event**.

(2) **Bloc** nhận Event → Xử lý → Phát ra **State** mới qua Stream.

(3) UI lắng nghe Stream → Nhận State → Cập nhật giao diện.

Ưu điểm của BLoC là tính nhất quán cao, dễ dàng viết Unit Test cho từng luồng sự kiện, và code rất rõ ràng. Tuy nhiên, nhược điểm là cần viết nhiều code mẫu (boilerplate) cho Event và State classes.

4.8. So sánh và Lựa chọn giải pháp

Không có giải pháp nào là "tốt nhất" cho mọi trường hợp. Việc lựa chọn phụ thuộc vào quy mô dự án và kinh nghiệm của đội ngũ.

Tiêu chí	**Provider**	**Riverpod**	**BLoC**	**GetX**
Độ khó	Thấp	Trung bình	Cao	Thấp
Code mẫu	Ít	Trung bình	Nhiều	Rất ít
Phụ thuộc Context	Có	Không	Có (BlocProvider)	Không
Khả năng kiểm thử	Trung bình	Tốt	Rất tốt	Khó
Phù hợp	App nhỏ/trung bình	App mọi quy mô	App doanh nghiệp	App cần code nhanh (MVP)

Lời khuyên:

- Nếu chúng ta mới học Flutter: Hãy bắt đầu với **Provider**.
- Nếu làm dự án mới nghiêm túc: Hãy cân nhắc **Riverpod**.
- Nếu làm việc trong môi trường Enterprise yêu cầu chuẩn hóa cao: **BLoC** là lựa chọn an toàn.

4.9. Best Practices

Để quản lý trạng thái hiệu quả, hãy tuân thủ các nguyên tắc sau:

- **Tính bất biến (Immutability):** State nên là bất biến. Khi cần thay đổi, hãy tạo một đối tượng mới thay vì sửa đổi đối tượng cũ. Sử dụng từ khóa final và phương thức copyWith. Điều này giúp dễ dàng so sánh trạng thái cũ và mới để tối ưu rebuild.
- **Single Source of Truth (Một nguồn dữ liệu duy nhất):** Dữ liệu chỉ nên được lưu trữ ở một nơi duy nhất. Tránh việc lưu cùng một biến trạng thái ở nhiều nơi khác nhau gây mâu thuẫn dữ liệu.
- **Tách biệt quan tâm (Separation of Concerns):** Widget chỉ nên làm nhiệm vụ hiển thị. Không gọi API trực tiếp trong Widget. Hãy gọi API từ Repository hoặc Service thông qua State Management.
- **Quản lý bộ nhớ:** Luôn giải phóng tài nguyên (dispose) khi Widget bị hủy. Ví dụ: đóng StreamController, hủy listener để tránh rò rỉ bộ nhớ (memory leak).
- **Tránh rebuild không cần thiết:** Chỉ bao quanh những Widget cần cập nhật bằng Consumer. Đừng bao quanh toàn bộ Scaffold nếu chỉ có một chữ Text thay đổi. Sử dụng Selector (trong Provider) để lắng nghe cụ thể một trường dữ liệu.

4.10. Các lỗi thường gặp (Pitfalls)

- **Rebuild quá mức:** Đặt Consumer bao quanh cây Widget quá lớn khiến toàn bộ màn hình vẽ lại khi chỉ một dữ liệu nhỏ thay đổi.
- **Sai Context:** Truy cập Provider sau khi Widget đã bị hủy (dispose), hoặc dùng context của cha khi cần context của con trong cây Provider.
- **Logic trong UI:** Viết các xử lý nghiệp vụ phức tạp ngay trong hàm onPressed của nút bấm thay vì đưa vào Model/Bloc.
- **Quên notifyListeners:** Thay đổi dữ liệu trong Model nhưng quên gọi hàm này, dẫn đến UI không cập nhật dù dữ liệu đã đổi.

4.11. Bài tập thực hành

Để củng cố kiến thức, hãy thực hiện các bài tập sau:

Bài tập cơ bản:

- Tạo ứng dụng Counter mặc định của Flutter.
- Refactor (tái cấu trúc) lại ứng dụng đó, chuyển từ dùng setState sang dùng **Provider**.

Bài tập trung bình:

- Xây dựng màn hình Đăng nhập (Login).
- Lưu trạng thái "Đã đăng nhập" vào App State.
- Nếu trạng thái là đã đăng nhập, điều hướng vào Trang Chủ. Nếu chưa, giữ ở Trang Login.
- Dữ liệu đăng nhập phải được chia sẻ để hiển thị tên user ở góc màn hình Trang Chủ.

Bài tập nâng cao:

- Xây dựng ứng dụng Todo List (Quản lý công việc).
- Chức năng: Thêm, Xóa, Đánh dấu hoàn thành.
- Yêu cầu: Sử dụng **BLoC** hoặc **Riverpod**.
- Dữ liệu phải được lưu trữ tạm thời và không mất đi khi chuyển đổi qua lại giữa các màn hình.

4.12. Tổng kết

Quản lý trạng thái là kỹ năng quan trọng nhất để chuyển từ một người biết viết code Flutter sang một kỹ sư phát triển ứng dụng chuyên nghiệp.

- Hiểu rõ sự khác biệt giữa Ephemeral State và App State.
- Biết khi nào nên dùng setState và khi nào cần dùng thư viện bên ngoài.
- Thành thạo ít nhất một giải pháp (khuyên dùng Provider hoặc Riverpod) và hiểu nguyên lý của các giải pháp khác.

Việc lựa chọn đúng công cụ quản lý trạng thái ngay từ đầu sẽ tiết kiệm rất nhiều thời gian bảo trì và mở rộng tính năng về sau.

Chương 5: Layout & Responsive UI

5.1. Giới thiệu

Trong phát triển ứng dụng di động, giao diện người dùng (UI) là yếu tố đầu tiên và quan trọng nhất quyết định trải nghiệm của người dùng. Nếu logic nghiệp vụ là "phần hồn" của ứng dụng, thì Layout (bố cục) chính là "phần xác" mà người dùng tương tác trực tiếp.

Khác với phát triển Web truyền thống sử dụng HTML và CSS để dàn trang, Flutter tiếp cận việc xây dựng giao diện hoàn toàn bằng code thông qua hệ thống **Widget**. Chương này sẽ cung cấp cho chúng ta kiến thức nền tảng về cơ chế Layout của Flutter, cách sử dụng các widget bố cục cơ bản, và quan trọng nhất là kỹ thuật xây dựng giao diện thích ứng (Responsive UI) để ứng dụng của chúng ta có thể chạy mượt mà trên nhiều kích thước màn hình khác nhau từ điện thoại nhỏ đến máy tính bảng hay web.

5.2. Triết lý Layout của Flutter

Để làm chủ được giao diện trong Flutter, chúng ta cần hiểu rõ cách Flutter quyết định vị trí và kích thước của các thành phần trên màn hình. Flutter không sử dụng CSS, do đó mọi thứ đều được kiểm soát bởi chính các Widget.

5.2.1. Mọi thứ đều là Widget

Trong Flutter, từ một nút bấm, đoạn văn bản, cho đến khoảng trống hay cả màn hình đều là Widget. Giao diện được xây dựng dưới dạng một cây phân cấp gọi là **Widget Tree**. Widget cha sẽ chứa và quyết định hành vi của widget con.

5.2.2. Cơ chế 3 bước (The 3-Step Mechanism)

Quá trình render layout trong Flutter diễn ra theo một quy trình chặt chẽ giữa Widget cha (Parent) và Widget con (Child), bao gồm 3 bước chính:

- **Constraints (Ràng buộc):** Widget cha gửi xuống cho widget con các giới hạn về kích thước (ví dụ: chiều rộng tối thiểu, tối đa, chiều cao tối thiểu, tối đa).
- **Size (Kích thước):** Widget con chọn một kích thước cụ thể nằm trong khoảng ràng buộc mà cha đã gửi và báo cáo ngược lên cho cha.
- **Position (Vị trí):** Widget cha quyết định vị trí hiển thị của widget con dựa trên kích thước mà con đã chọn.

Lưu ý quan trọng: Widget con **không thể** tự quyết định kích thước của nó. Nó chỉ có thể chọn kích thước nằm trong phạm vi cho phép của cha. Ví dụ: Một Container không thể rộng hơn màn hình nếu cha của nó (thường là Scaffold hoặc Material) chỉ cho phép nó rộng bằng màn hình.

5.3. Các Widget Layout Cơ Bản

Flutter cung cấp một thư viện phong phú các widget để dàn trang. Chúng ta có thể chia chúng thành hai nhóm chính: Widget đơn con (Single Child) và Widget đa con (Multi-Child).

5.3.1. Widget đơn con (Single Child Widgets)

Những widget này chỉ chứa một widget con duy nhất, thường dùng để định dạng hoặc bao bọc.

- **Container:** Đây là widget đa năng nhất. Nó cho phép vẽ một hộp chữ nhật có thể tùy chỉnh màu sắc, viền (border), bóng (shadow), khoảng cách lề (margin), khoảng cách đệm (padding) và kích thước.

- **Padding:** Dùng để tạo khoảng trống bên trong (giữa viền và nội dung).
- **Center:** Căn giữa widget con theo cả chiều ngang và dọc.
- **Align:** Căn widget con vào một vị trí cụ thể (ví dụ: Alignment.topLeft, Alignment.bottomRight).
- **SizedBox:** Dùng để ép buộc kích thước cố định cho widget con hoặc tạo khoảng trống vô hình. SizedBox nhẹ hơn Container nên hãy ưu tiên dùng khi chỉ cần kích thước cố định.

Ví dụ:

```
Container(
  margin: EdgeInsets.all(10.0), // Khoảng cách
ngoài
  padding: EdgeInsets.all(5.0), // Khoảng cách
trong
  color: Colors.blueAccent,
  child: Center(
    child: Text('Xin chào Flutter'),
  ),
)
```

5.3.2. Widget đa con (Multi-Child Widgets)

Những widget này cho phép chứa nhiều widget con và sắp xếp chúng theo các quy tắc nhất định.

Row và Column

Đây là hai widget phổ biến nhất để dàn trang.

- **Row:** Sắp xếp các con theo chiều **ngang**.
- **Column:** Sắp xếp các con theo chiều **dọc**.

Hai widget này có hai thuộc tính căn chỉnh quan trọng:

- **MainAxisAlignment:** Căn chỉnh dọc theo trục chính (Row là ngang, Column là dọc). Các giá trị thường dùng: start, center, end, spaceBetween, spaceAround.

- **CrossAxisAlignment:** Căn chỉnh dọc theo trục phụ (vuông góc với trục chính).

Cảnh báo: Row không thể chứa một widget có chiều cao vô hạn (như một Column không bị giới hạn), và ngược lại. Nếu gặp lỗi này, chúng ta cần bọc widget con trong Expanded hoặc SizedBox.

Stack và Positioned

Khi chúng ta muốn các widget chồng lên nhau (ví dụ: một icon nằm trên góc của bức ảnh), hãy dùng **Stack**.

- **Stack:** Xếp chồng các widget con lên nhau theo thứ tự khai báo (widget sau nằm trên widget trước).
- **Positioned:** Dùng bên trong Stack để định vị widget con dựa trên các cạnh (top, bottom, left, right).

Ví dụ:

```
Stack(
  children: [

Image.network('https://via.placeholder.com/150'),
    Positioned(
      bottom: 0,
      right: 0,
      child: Icon(Icons.favorite, color:
Colors.red, size: 30),
    ),
  ],
)
```

5.3.3. Flexible và Expanded

Trong một Row hoặc Column, đôi khi chúng ta muốn các widget con tự động chiếm không gian còn lại thay vì kích thước cố định.

- **Expanded:** Widget con sẽ bắt buộc chiếm hết không gian còn lại. Tương đương với Flexible(fit: FlexFit.tight).

- **Flexible:** Widget con có thể chiếm không gian còn lại nhưng không bắt buộc (có thể nhỏ hơn nếu nội dung ít).

Chúng ta có thể sử dụng thuộc tính flex để chia tỷ lệ không gian. Ví dụ: flex: 2 sẽ chiếm không gian gấp đôi flex: 1.

```
Row(
  children: [
    Expanded(flex: 2, child: Container(color:
Colors.red)),   // 2/3 màn hình
    Expanded(flex: 1, child: Container(color:
Colors.blue)),  // 1/3 màn hình
  ],
)
```

5.4. Xây dựng Responsive UI

Một thách thức lớn trong phát triển đa nền tảng là ứng dụng phải hiển thị tốt trên nhiều kích thước màn hình khác nhau (iPhone nhỏ, Android lớn, iPad, Web...). Giao diện thích ứng (Responsive UI) giải quyết vấn đề này.

5.4.1. Sử dụng MediaQuery

MediaQuery cho phép chúng ta lấy thông tin về màn hình thiết bị đang chạy ứng dụng (chiều rộng, chiều cao, độ sáng, hướng màn hình...).

Chúng ta có thể tính toán kích thước widget dựa trên tỷ lệ phần trăm của màn hình.

```
var screenWidth =
MediaQuery.of(context).size.width;
var containerWidth = screenWidth * 0.5;
// Chiếm 50% chiều rộng màn hình
Container(
  width: containerWidth,
  child: Text('Responsive Width'),
```

```
)
```

Nhược điểm: MediaQuery không tự động rebuild khi kích thước màn hình thay đổi động (ví dụ: khi người dùng chia đôi màn hình trên Android mà không có cấu hình thêm).

5.4.2. Sử dụng LayoutBuilder

LayoutBuilder là công cụ mạnh mẽ hơn cho responsive. Nó cung cấp thông tin BoxConstraints từ widget cha ngay tại thời điểm render. Chúng ta có thể quyết định hiển thị widget nào dựa trên không gian thực tế available.

Ví dụ: Chuyển từ giao diện 1 cột (Mobile) sang 2 cột (Tablet).

```
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 600) {
      // Màn hình rộng (Tablet/Desktop)
      return Row(children: [Menu(), Content()]);
    } else {
      // Màn hình hẹp (Mobile)
      return Column(children: [Menu(),
Content()]);
    }
  },
)
```

5.4.3. OrientationBuilder và Wrap

- **OrientationBuilder:** Giúp phát hiện thay đổi hướng màn hình (Portrait - dọc vs Landscape - ngang). Hữu ích khi chúng ta muốn thay đổi layout khi người dùng xoay điện thoại.
- **Wrap:** Giống như Row hoặc Column nhưng có khả năng tự động xuống dòng khi không còn chỗ. Rất hữu ích cho các danh sách tag, chip, hoặc nút bấm.

```
Wrap(
  spacing: 8.0,
```

```
// Khoảng cách giữa các item trên cùng dòng
  runSpacing: 4.0, // Khoảng cách giữa các dòng
  children: [
    Chip(label: Text('Flutter')),
    Chip(label: Text('Dart')),
    Chip(label: Text('Mobile')),
    Chip(label: Text('Web')),
  ],
)
```

5.4.4. AspectRatio và FittedBox

- **AspectRatio:** Giữ tỷ lệ khung hình cố định cho widget con (ví dụ: 16:9 cho video, 1:1 cho ảnh đại diện) bất kể kích thước màn hình.
- **FittedBox:** Co giãn widget con (thường là Text hoặc Icon) để vừa khít với widget cha mà không bị vỡ cấu trúc.

5.5. Scrollable Layouts

Nội dung ứng dụng thường dài hơn kích thước màn hình. Flutter cung cấp các widget hỗ trợ cuộn (scroll):

- **ListView:** Hiển thị danh sách theo chiều dọc hoặc ngang.
  - Nên dùng ListView.builder khi danh sách dài để tối ưu hiệu năng (chỉ render các item visible).
- **GridView:** Hiển thị danh sách dạng lưới.
  - GridView.count: Dễ dùng khi muốn cố định số cột.
  - GridView.builder: Linh hoạt hơn cho dữ liệu động.
- **SingleChildScrollView:** Dùng khi chúng ta chỉ có một widget lớn duy nhất cần cuộn (ví dụ: một Form dài).

Lỗi thường gặp: Không thể đặt ListView hoặc GridView bên trong Column mà không bọc nó trong Expanded. Lý do là Column muốn con

của nó có kích thước vô hạn, nhưng ListView cũng muốn chiều cao vô hạn để cuộn -> Xung đột ràng buộc.

5.6. Xử lý lỗi Layout thường gặp

Khi làm việc với layout, chúng ta sẽ gặp một số lỗi đặc trưng. Hiểu rõ chúng sẽ giúp chúng ta debug nhanh hơn.

5.6.1. RenderFlex Overflow (Sọc vàng đen)

- **Hiện tượng:** Xuất hiện các sọc vàng đen ở mép màn hình kèm thông báo lỗi trên Console.
- **Nguyên nhân:** Widget con quá lớn so với không gian mà widget cha (Row/Column) cung cấp.
- Cách sửa:
  - Dùng Expanded hoặc Flexible để cho phép widget con co giãn.
  - Dùng SingleChildScrollView để cho phép cuộn thay vì cố định kích thước.
  - Dùng Wrap để tự động xuống dòng.
  - Giảm kích thước cố định của widget con.

5.6.2. BoxConstraints forces an infinite size

- **Hiện tượng:** Lỗi runtime đỏ màn hình.
- **Nguyên nhân:** Widget cha không giới hạn kích thước (vô hạn), và widget con cũng không tự giới hạn kích thước của nó.
- **Cách sửa:** Gán kích thước cụ thể (width/height) hoặc bọc widget con trong Expanded để nó nhận ràng buộc từ cha.

5.6.3. Hiệu năng kém (Slow Animation)

- **Nguyên nhân:** Cây widget quá sâu, hoặc rebuild toàn bộ cây khi chỉ một phần nhỏ thay đổi.
- Cách sửa:
  - Tách nhỏ widget thành các class riêng.

- Sử dụng từ khóa const cho các widget không đổi.
- Sử dụng Key phù hợp khi dùng danh sách động.

5.7. Best Practices

Để mã nguồn sạch sẽ, dễ bảo trì và hiệu năng cao, hãy tuân thủ các nguyên tắc sau:

- **Tách nhỏ Widget:** Đừng viết toàn bộ giao diện trong hàm build của một màn hình. Hãy tách các phần tử lặp lại hoặc phức tạp thành các Widget riêng biệt. Điều này giúp dễ đọc code và tối ưu rebuild.
- **Ưu tiên LayoutBuilder:** Khi làm responsive, LayoutBuilder thường linh hoạt và chính xác hơn MediaQuery vì nó dựa trên không gian thực tế của cha chứ không phải kích thước toàn màn hình.
- **Tránh Hard-code kích thước:** Hạn chế ghi cứng width: 300. Hãy dùng tỷ lệ phần trăm, Expanded, hoặc MediaQuery để giao diện linh hoạt.
- **Sử dụng const:** Luôn dùng const cho các widget không thay đổi trạng thái để giảm thiểu việc tạo lại object không cần thiết.
- **Test đa thiết bị:** Luôn kiểm tra giao diện trên nhiều kích thước màn hình khác nhau (sử dụng Flutter DevTools hoặc các emulator khác nhau) trước khi hoàn thiện.

5.8. Bài tập thực hành

Bài tập 1 (Cơ bản): Xây dựng màn hình Login đơn giản. Bao gồm Logo ở giữa, 2 ô nhập liệu (Email, Password) và nút Login. Sử dụng Column, Container, Padding.

Bài tập 2 (Responsive): Xây dựng màn hình sản phẩm.

- Trên điện thoại: Hiển thị danh sách sản phẩm 2 cột.
- Trên máy tính bảng: Hiển thị danh sách sản phẩm 4 cột.

- Sử dụng LayoutBuilder hoặc GridView.count kết hợp MediaQuery.

Bài tập 3 (Debug): Cố tình tạo một lỗi RenderFlex Overflow bằng cách đặt một Container quá rộng vào Row. Sau đó tìm cách sửa lỗi nó bằng Expanded hoặc SingleChildScrollView.

5.9. Tổng kết

Chương 5 đã cung cấp cho chúng ta cái nhìn toàn diện về hệ thống Layout trong Flutter. Từ việc hiểu cơ chế ràng buộc (constraints) đến việc sử dụng thành thạo các widget như Row, Column, Stack, và các công cụ responsive như LayoutBuilder, MediaQuery.

Layout là kỹ năng cốt lõi của lập trình viên Flutter. Một giao diện đẹp, mượt mà và thích ứng tốt sẽ là nền tảng vững chắc cho sự thành công của ứng dụng. Ở các chương tiếp theo, chúng ta sẽ kết hợp những kiến thức layout này với việc xử lý sự kiện người dùng và gọi API để tạo nên những ứng dụng hoàn chỉnh.

Chương 6: Navigation & Routing

6.1. Giới thiệu

Trong phát triển ứng dụng di động, hiếm khi một ứng dụng chỉ có duy nhất một màn hình. Hầu hết các ứng dụng thực tế đều bao gồm nhiều màn hình khác nhau như: Màn hình chào mừng, Đăng nhập, Trang chủ, Chi tiết sản phẩm, Cài đặt, v.v. Việc chuyển đổi giữa các màn hình này một cách mượt mà và quản lý luồng dữ liệu giữa chúng là nhiệm vụ của **Navigation & Routing**.

Flutter cung cấp một hệ thống điều hướng mạnh mẽ dựa trên nguyên lý ngăn xếp (Stack). Chương này sẽ giúp chúng ta hiểu rõ cơ chế hoạt động, cách thực hiện điều hướng cơ bản, truyền dữ liệu giữa các màn hình và các phương pháp quản lý route chuyên nghiệp.

6.2. Cơ chế Navigation Stack

Để hiểu cách Flutter điều hướng, chúng ta cần nắm vững khái niệm **Stack (Ngăn xếp)**. Flutter quản lý các màn hình (Route) như các lớp chồng lên nhau.

6.2.1. Nguyên lý LIFO (Last In, First Out)

- **Màn hình hiện tại** luôn nằm trên cùng của ngăn xếp.
- Khi chúng ta chuyển đến một màn hình mới, nó được **đẩy (push)** lên trên cùng.
- Khi chúng ta quay lại, màn hình hiện tại bị **loại bỏ (pop)** khỏi ngăn xếp, để lộ màn hình nằm ngay bên dưới nó.

6.3. Điều hướng cơ bản (Imperative Navigation)

Cách đơn giản nhất để điều hướng trong Flutter là sử dụng trực tiếp class Navigator. Phương thức này còn gọi là điều hướng mệnh lệnh (Imperative).

6.3.1. Chuyển đến màn hình mới (Push)

Để mở một màn hình mới, ta sử dụng Navigator.push. Phương thức này cần một BuildContext và một Route (thường là MaterialPageRoute).

```
// Từ Màn hình A, muốn mở Màn hình B
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) =>
ScreenB()),
);
```

- context: Vị trí của widget trong cây widget, giúp Navigator biết nó đang hoạt động ở đâu.
- MaterialPageRoute: Tạo hiệu ứng chuyển cảnh trượt ngang (slide) phù hợp với chuẩn thiết kế Material (Android) và Cupertino (iOS).

6.3.2. Quay lại màn hình cũ (Pop)

Để đóng màn hình hiện tại và quay về màn hình trước đó, ta sử dụng Navigator.pop.

```
// Trong Màn hình B, muốn quay về Màn hình A
Navigator.pop(context);
```

Thông thường, lệnh này được gắn vào nút "Back" trên thanh AppBar hoặc một nút bấm tùy chỉnh.

6.4. Truyền dữ liệu giữa các màn hình

Trong thực tế, các màn hình không tồn tại độc lập mà cần trao đổi dữ liệu.

6.4.1. Truyền dữ liệu xuôi (Forward Data)

Truyền dữ liệu từ màn hình trước (A) sang màn hình sau (B). Cách phổ biến nhất là truyền qua **Constructor**.

Ví dụ: Truyền thông tin sản phẩm từ danh sách vào trang chi tiết.

```
// Màn hình A (Danh sách)
class Product {
  final String name;
  final double price;
  Product(this.name, this.price);
}

// Khi bấm vào sản phẩm
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => DetailScreen(product:
selectedProduct),
  ),
);

// Màn hình B (Chi tiết)
class DetailScreen extends StatelessWidget {
  final Product product; // Nhận dữ liệu

  // Constructor nhận dữ liệu
  DetailScreen({required this.product});

  @override
  Widget build(BuildContext context) {
    return Text('Sản phẩm: ${product.name}');
  }
}
```

6.4.2. Truyền dữ liệu ngược (Backward Data)

Truyền dữ liệu từ màn hình sau (B) trả về cho màn hình trước (A). Ví dụ: Chọn ảnh từ thư viện, đăng nhập thành công, hoặc xác nhận một hành động.

Cơ chế này dựa trên Future. Khi push một màn hình, nó trả về một Future sẽ hoàn thành khi màn hình đó đóng.

Bước 1: Màn hình A chờ kết quả Sử dụng từ khóa await để đợi màn hình B đóng và trả dữ liệu.

```
// Trong Màn hình A
final result = await Navigator.push(
  context,
  MaterialPageRoute(builder: (context) =>
ScreenB()),
);

print('Dữ liệu nhận từ B: $result');
```

Bước 2: Màn hình B gửi kết quả Truyền giá trị vào hàm Navigator.pop.

```
// Trong Màn hình B
// Khi người dùng bấm nút xác nhận
Navigator.pop(context, "Dữ liệu đã xử lý xong");
```

Lưu ý: Nếu không có await ở màn hình A, code sẽ chạy tiếp ngay lập tức mà không chờ màn hình B đóng.

6.5. Named Routes (Định tuyến có tên)

Khi ứng dụng lớn lên, việc sử dụng MaterialPageRoute rải rác khắp nơi khiến code khó bảo trì và khó thay đổi. **Named Routes** giải quyết vấn đề này bằng cách đặt tên cho các màn hình và cấu hình tập trung.

6.5.1. Cấu hình Route

Khai báo các route trong widget MaterialApp gốc của ứng dụng.

```
void main() {
  runApp(MaterialApp(
    initialRoute: '/', // Màn hình khởi động
    routes: {
      '/': (context) => HomeScreen(),
```

```
        '/detail': (context) => DetailScreen(),
        '/settings': (context) => SettingsScreen(),
      },
    ));
  }
```

6.5.2. Sử dụng Named Routes

Thay vì tạo MaterialPageRoute, ta chỉ cần gọi tên route.

```
// Chuyển màn hình
Navigator.pushNamed(context, '/detail');

// Quay lại
Navigator.pop(context);
```

6.5.3. Truyền tham số với Named Routes

Vì không dùng Constructor để truyền dữ liệu trực tiếp, ta sử dụng thuộc tính arguments.

Gửi dữ liệu:

```
Navigator.pushNamed(
  context,
  '/detail',
  arguments: {'id': 123, 'name': 'Flutter Book'},
);
```

Nhận dữ liệu: Tại màn hình đích, truy cập qua ModalRoute.

```
class DetailScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // Lấy arguments và ép kiểu (cast)
    final args =
ModalRoute.of(context)!.settings.arguments as
Map;

    return Text('ID: ${args['id']}');
  }
```

```
}
```

Cảnh báo: Việc ép kiểu as Map không an toàn về kiểu dữ liệu (type-safe). Nếu truyền sai kiểu, ứng dụng sẽ báo lỗi runtime. Trong các dự án lớn, nên tạo một class riêng để đóng gói arguments.

6.6. Các phương thức điều hướng nâng cao

Tùy vào ngữ cảnh nghiệp vụ, việc đẩy vào stack hay thay thế stack sẽ khác nhau.

Phương thức	**Mô tả**	**Trường hợp sử dụng**
push	Thêm màn hình mới vào đỉnh stack.	Xem chi tiết, mở cài đặt phụ.
pop	Xóa màn hình hiện tại khỏi stack.	Nút Back, Hủy bỏ.
pushReplacement	Xóa màn hình hiện tại, thay bằng màn hình mới.	**Đăng nhập thành công** (Không cho quay lại màn hình Login).
pushAndRemoveUntil	Xóa toàn bộ stack cũ, chỉ giữ màn hình mới.	**Đăng xuất (Logout)**, Reset ứng dụng về màn hình chính.

Ví dụ: Đăng xuất (Logout) Khi người dùng đăng xuất, ta muốn xóa toàn bộ lịch sử điều hướng để họ không thể bấm Back quay lại trang cá nhân.

```
Navigator.pushAndRemoveUntil(
  context,
  MaterialPageRoute(builder: (context) =>
LoginScreen()),
  (route) => false, // Xóa tất cả các route cũ
```

```
);
```

6.7. Xu hướng hiện đại: go_router package

Kể từ Flutter 2.0, cơ chế điều hướng đã phát triển lên phiên bản **Navigator 2.0**, cho phép quản lý state của route declaratively (khai báo). Tuy nhiên, API gốc của Navigator 2.0 khá phức tạp.

Để đơn giản hóa, cộng đồng và Google khuyến nghị sử dụng **go_router** packahe.

Ưu điểm của go_router:

- **URL-based:** Quản lý route giống như web (ví dụ: /home, /user/123).
- **Deep Linking:** Hỗ trợ mở ứng dụng từ link bên ngoài dễ dàng.
- **Type-safe:** Giảm thiểu lỗi khi truyền tham số.
- **Nested Navigation:** Hỗ trợ điều hướng lồng ghép phức tạp (ví dụ: Bottom Navigation Bar kết hợp Stack).

Ví dụ cấu hình cơ bản:

```
final GoRouter _router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (context, state)
=> HomeScreen()),
    GoRoute(path: '/detail', builder: (context,
state) => DetailScreen()),
  ],
);

// Điều hướng
context.go('/detail');
```

6.8. Bài tập thực hành

Yêu cầu: Xây dựng ứng dụng quản lý sinh viên đơn giản gồm 3 màn hình.

- Màn hình Login:
  - Có 2 ô nhập liệu (Username, Password).
  - Nút "Đăng nhập".
  - *Logic:* Nếu đăng nhập thành công, chuyển sang màn hình Home và **không cho phép quay lại** màn hình Login.
- Màn hình Home (Danh sách):
  - Hiển thị danh sách tên sinh viên (Hardcode một danh sách giả).
  - Khi bấm vào một sinh viên, chuyển sang màn hình Detail.
  - Có nút "Đăng xuất" ở góc màn hình.
  - *Logic:* Khi đăng xuất, quay về màn hình Login và **xóa toàn bộ lịch sử điều hướng**.
- Màn hình Detail (Chi tiết):
  - Hiển thị thông tin chi tiết của sinh viên được chọn (Tên, Lớp, Điểm).
  - Có nút "Sửa điểm". Khi bấm vào, hiện một hộp thoại nhập điểm mới.
  - *Logic:* Khi lưu điểm mới, quay lại màn hình Home và **cập nhật** điểm của sinh viên đó trong danh sách (Sử dụng truyền dữ liệu ngược).

Gợi ý:

- Sử dụng StatefulWidget cho màn hình Home để quản lý danh sách sinh viên có thể thay đổi.
- Sử dụng pushReplacement cho luồng Đăng nhập.
- Sử dụng pushAndRemoveUntil cho luồng Đăng xuất.

6.9. Tổng kết

Điều hướng là xương sống của trải nghiệm người dùng trong ứng dụng di động. Những điểm cần ghi nhớ:

- Flutter sử dụng **Stack** để quản lý màn hình.
- push để thêm, pop để bớt.
- Dữ liệu xuôi truyền qua Constructor, dữ liệu ngược truyền qua await và pop(value).
- Sử dụng **Named Routes** để code sạch hơn và dễ bảo trì.
- Chọn đúng phương thức (push vs replacement) để đảm bảo luồng nghiệp vụ đúng (đặc biệt là Login/Logout).

Chương 7: Style & Theme

7.1. Giới thiệu

Trong quá trình phát triển ứng dụng di động, giao diện người dùng (UI) đóng vai trò then chốt trong việc tạo ấn tượng và trải nghiệm cho người dùng. Một ứng dụng chuyên nghiệp không chỉ cần chức năng đúng đắn mà còn phải có giao diện nhất quán, đẹp mắt và dễ nhìn. Flutter cung cấp một hệ thống mạnh mẽ để quản lý giao diện thông qua hai khái niệm chính: **Style** (Kiểu dáng) và **Theme** (Chủ đề).

Chương này sẽ hướng dẫn chúng ta cách phân biệt và sử dụng linh hoạt Style và Theme để xây dựng giao diện ứng dụng đồng bộ, dễ bảo trì và hỗ trợ đa chế độ hiển thị (Dark Mode/Light Mode).

7.2. Khái niệm Style và Theme

Để quản lý giao diện hiệu quả, trước hết cần hiểu rõ sự khác biệt giữa hai khái niệm nền tảng sau:

7.2.1. Style (Kiểu dáng cục bộ)

Style là tập hợp các thuộc tính trang trí áp dụng cho một **Widget cụ thể**. Nó mang tính chất cục bộ (Local).

- **Ví dụ:** Chúng ta muốn một dòng chữ cụ thể có màu đỏ, in đậm, kích thước 20px. Chúng ta sẽ sử dụng TextStyle ngay trong widget Text đó.
- **Ưu điểm:** Linh hoạt, dễ thay đổi nhanh cho một vị trí đặc biệt.
- **Nhược điểm:** Khó bảo trì khi ứng dụng lớn. Nếu muốn đổi màu đỏ thành màu xanh cho 100 vị trí, chúng ta phải sửa code ở 100 chỗ.

7.2.2. Theme (Chủ đề toàn cục)

Theme là tập hợp các quy tắc style được định nghĩa một lần và áp dụng cho **toàn bộ ứng dụng**. Nó mang tính chất toàn cục (Global).

- **Ví dụ:** Chúng ta định nghĩa màu chủ đạo (Primary Color) của ứng dụng là màu Xanh dương. Tất cả các nút bấm, thanh tiêu đề, hiệu ứng nhấn... mặc định sẽ sử dụng màu này.
- **Ưu điểm:** Tính nhất quán cao, dễ bảo trì, dễ dàng thay đổi giao diện toàn bộ app chỉ bằng vài dòng code.
- **Cơ chế:** Flutter sử dụng cơ chế InheritedWidget để truyền dữ liệu Theme từ gốc xuống toàn bộ cây Widget.

7.3. Styling cơ bản (Inline Style)

Trong Flutter, hầu hết các Widget đều có thuộc tính style hoặc các thuộc tính liên quan đến giao diện. Cách đơn giản nhất là khai báo trực tiếp (Inline).

Ví dụ: Tùy chỉnh một widget Text.

```
Text(
  'Xin chào Flutter',
  style: TextStyle(
    fontSize: 24,             // Kích thước chữ
    fontWeight: FontWeight.bold, // Độ đậm
    color: Colors.blue,       // Màu sắc
    decoration: TextDecoration.underline, // Gạch
chân
    letterSpacing: 1.5,       // Khoảng cách giữa
các ký tự
    height: 1.5,              // Chiều cao dòng
  ),
)
```

Ví dụ: Tùy chỉnh một widget Container.

```
Container(
  padding: EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [
```

```
        BoxShadow(
          color: Colors.grey.withOpacity(0.5),
          spreadRadius: 2,
          blurRadius: 5,
        ),
      ],
    ),
    child: Text('Nội dung box'),
  )
```

Lưu ý: Mặc dù dễ sử dụng, chúng ta nên hạn chế lạm dụng Inline Style cho các thành phần lặp lại nhiều lần. Hãy ưu tiên sử dụng Theme để chuẩn hóa.

7.4. Theme toàn cục với ThemeData

Để thiết lập Theme cho toàn bộ ứng dụng, chúng ta làm việc với widget cấp cao nhất thường là MaterialApp (đối với ứng dụng theo chuẩn Material Design) hoặc CupertinoApp (chuẩn iOS).

7.4.1. Cấu hình ThemeData

Đối tượng ThemeData chứa tất cả các cấu hình về màu sắc, font chữ, hình dáng widget...

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Theme Demo',
      // Cấu hình Theme toàn cục
```

```
      theme: ThemeData(
        // Material 3 khuyến khích dùng
colorScheme
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
          brightness: Brightness.light,
        ),
        // Font chữ mặc định cho toàn app
        fontFamily: 'Roboto',
        // Cấu hình cụ thể cho AppBar
        appBarTheme: const AppBarTheme(
          centerTitle: true,
          elevation: 0,
        ),
        // Cấu hình cụ thể cho ElevatedButton
        elevatedButtonTheme:
ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            shape: RoundedRectangleBorder(
              borderRadius:
BorderRadius.circular(10),
            ),
          ),
        ),
      ),
      home: const MyHomePage(),
    );
  }
}
```

7.4.2. Các thuộc tính quan trọng trong ThemeData

- **colorScheme**: Định nghĩa bảng màu toàn cục (primary, secondary, surface, error, background...). Đây là cách tiếp cận hiện đại nhất (Material 3).

- **textTheme**: Định nghĩa các kiểu chữ chuẩn (headlineLarge, bodyMedium, labelSmall...). Khi chúng ta dùng Text mà không truyền style, nó sẽ lấy từ đây.
- **inputDecorationTheme**: Chuẩn hóa giao diện cho TextField (viền, màu label, màu khi focus...).
- **scaffoldBackgroundColor**: Màu nền mặc định cho các màn hình Scaffold.
- **primarySwatch**: (Cũ - Material 2) Thường dùng để định nghĩa màu chủ đạo nhanh.

7.5. Truy cập dữ liệu Theme trong Widget

Khi đang xây dựng các Widget con, làm thế nào để lấy được màu sắc hoặc kiểu chữ từ Theme toàn cục? Flutter cung cấp phương thức Theme.of(context).

7.5.1. Cách sử dụng

```
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // Truy cập đối tượng Theme
    final theme = Theme.of(context);

    // Lấy màu chủ đạo
    final primaryColor =
theme.colorScheme.primary;

    // Lấy kiểu chữ body mặc định
    final bodyStyle = theme.textTheme.bodyLarge;

    return Scaffold(
      backgroundColor:
theme.scaffoldBackgroundColor,
      appBar: AppBar(
```

```
        title: Text('Demo Theme', style:
bodyStyle),
        backgroundColor: primaryColor,
      ),
      body: Center(
        child: ElevatedButton(
          // Nút sẽ tự động dùng màu primary từ
theme
          onPressed: () {},
          child: Text('Nút bấm'),
        ),
      ),
    );
  }
}
```

7.5.2. Lưu ý quan trọng

- **Context:** Biến context phải nằm trong cây widget con của MaterialApp thì mới truy cập được Theme.
- **Null Safety:** Một số thuộc tính trong textTheme có thể là nullable (ví dụ: bodyLarge), nên sử dụng toán tử ! hoặc ?? để xử lý an toàn.
- **Rebuild:** Khi Theme thay đổi, các widget sử dụng Theme.of(context) sẽ tự động rebuild lại để cập nhật giao diện mới.

7.6. Hỗ trợ Dark Mode và Light Mode

Một yêu cầu phổ biến hiện nay là ứng dụng phải hỗ trợ cả chế độ sáng (Light Mode) và chế độ tối (Dark Mode). Flutter hỗ trợ tính năng này rất dễ dàng thông qua MaterialApp.

7.6.1. Cấu hình đa chế độ

Chúng ta cần khai báo 2 đối tượng theme và một chế độ hoạt động:

```
MaterialApp(
```

```
    // Theme cho chế độ sáng
    theme: ThemeData(
      colorScheme: ColorScheme.fromSeed(
        seedColor: Colors.blue,
        brightness: Brightness.light,
      ),
    ),
    // Theme cho chế độ tối
    darkTheme: ThemeData(
      colorScheme: ColorScheme.fromSeed(
        seedColor: Colors.blue,
        brightness: Brightness.dark,
      ),
    ),
    // Chế độ hoạt động
    themeMode: ThemeMode.system, // Theo cài đặt
  của hệ thống điện thoại
    // Hoặc ThemeMode.light, ThemeMode.dark
    home: MyHomePage(),
  )
```

7.6.2. Chuyển đổi Theme động

Để cho phép người dùng tự bật/tắt Dark Mode trong ứng dụng, chúng ta cần quản trạng thái themeMode bằng StatefulWidget.

```
class MyApp extends StatefulWidget {
  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  ThemeMode _themeMode = ThemeMode.system;

  void toggleTheme(bool isDark) {
    setState(() {
      _themeMode = isDark ? ThemeMode.dark :
ThemeMode.light;
```

```
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      themeMode: _themeMode,
      theme: ThemeData.light(),
      darkTheme: ThemeData.dark(),
      home: MyHomePage(onToggleTheme:
toggleTheme),
    );
  }
}
```

7.7. Tùy chỉnh Font chữ và Assets

Sử dụng font chữ mặc định đôi khi không đủ để tạo nên bản sắc riêng cho ứng dụng. Flutter cho phép chúng ta nhúng các file font tùy thích.

7.7.1. Các bước thực hiện

- **Tải font:** Tải file font (đuôi .ttf hoặc .otf) về máy.
- **Tạo thư mục:** Tạo thư mục assets/fonts/ trong root dự án.
- Khai báo pubspec.yaml:

```
flutter:
  fonts:
    - family: Roboto
      fonts:
        - asset: assets/fonts/Roboto-Regular.ttf
        - asset: assets/fonts/Roboto-Bold.ttf
          weight: 700
    - family: OpenSans
      fonts:
        - asset: assets/fonts/OpenSans-
Regular.ttf
```

Lưu ý: *Indentation (thụt đầu dòng) trong pubspec.yaml rất quan trọng, phải đúng 2 khoảng trắng.*

- **Cập nhật code:** Chạy lệnh flutter pub get và sử dụng fontFamily trong ThemeData hoặc TextStyle.

7.7.2. Sử dụng package Google Fonts

Để tiết kiệm thời gian, chúng ta có thể sử dụng package google_fonts.

Thêm vào pubspec.yaml: `google_fonts: ^6.1.0`

Sử dụng:

```
import 'package:google_fonts/google_fonts.dart';

Text(
  'Xin chào',
  style: GoogleFonts.roboto(fontSize: 20),
)
```

7.8. Material Design vs Cupertino

Flutter hỗ trợ hai ngôn ngữ thiết kế chính:

- Material Design (Google):
  - Phù hợp cho Android, Web và iOS (nếu muốn giao diện đồng nhất).
  - Widget đặc trưng: Scaffold, AppBar, ElevatedButton, Card.
  - Sử dụng MaterialApp.
- Cupertino (Apple):
  - Phù hợp cho iOS muốn trải nghiệm native chuẩn Apple.
  - Widget đặc trưng: CupertinoPageScaffold, CupertinoNavigationBar, CupertinoButton.
  - Sử dụng CupertinoApp.

Khuyến nghị: *Trong phát triển đa nền tảng, đa số lập trình viên chọn* ***Material Design*** *làm chủ đạo vì tính linh hoạt và dễ tùy biến. Chúng ta vẫn có thể xen kẽ các widget Cupertino vào trong ứng dụng Material nếu cần thiết (ví dụ: CupertinoSwitch trên màn hình Material).*

7.9. Best Practices

Để code sạch và dễ bảo trì, hãy tuân thủ các nguyên tắc sau:

- **Không Hard-code màu sắc:** Tránh viết Colors.red trực tiếp trong UI. Hãy dùng Theme.of(context).colorScheme.error.
- **Tách biệt cấu hình:** Tạo một file riêng (ví dụ: app_theme.dart) để chứa cấu hình ThemeData.
- **Sử dụng Extension:** Tạo extension cho BuildContext để truy cập theme ngắn gọn hơn.

```
extension ContextExtension on BuildContext {
  Color get primaryColor =>
Theme.of(this).colorScheme.primary;
  TextTheme get textTheme =>
Theme.of(this).textTheme;
}
// Sử dụng: context.primaryColor
```

- **Constant Colors:** Nếu có màu thương hiệu đặc biệt không nằm trong theme, hãy khai báo trong file app_colors.dart dưới dạng static const.

7.10. Bài tập thực hành

Đề bài: Xây dựng màn hình Đăng nhập (Login Screen) với yêu cầu:

- Sử dụng ThemeData để định nghĩa màu chủ đạo là DeepPurple.
- Cấu hình textTheme để toàn bộ chữ trong app sử dụng font 'OpenSans' (có thể dùng google_fonts).
- Nút "Đăng nhập" sử dụng màu primary của Theme, bo tròn góc.

- Ô nhập liệu (TextField) sử dụng InputDecorationTheme để có viền thống nhất.
- **Nâng cao:** Thêm một Switch để người dùng bật/tắt Dark Mode ngay trên màn hình đăng nhập.

Gợi ý:

- Sử dụng StatefulWidget cho màn hình Login để quản lý trạng thái Switch.
- Truyền hàm callback từ màn hình Login lên MaterialApp để thay đổi themeMode.

7.11. Tổng kết

Chương 7 đã cung cấp cho chúng ta cái nhìn toàn diện về cách quản lý giao diện trong Flutter. Việc nắm vững **Style & Theme** không chỉ giúp ứng dụng của chúng ta đẹp hơn mà còn giảm thiểu đáng kể thời gian bảo trì code về sau.

Điểm cần ghi nhớ:

- Style dùng cho cục bộ, Theme dùng cho toàn cục.
- ThemeData là nơi khai báo cấu hình giao diện trung tâm.
- Theme.of(context) là cầu nối để Widget con truy cập dữ liệu giao diện.
- Luôn nghĩ đến việc hỗ trợ Dark Mode ngay từ khi bắt đầu dự án.

Chương 8: Quản lý Assets

8.1. Giới thiệu

Trong quá trình phát triển ứng dụng di động, giao diện người dùng (UI) và trải nghiệm người dùng (UX) đóng vai trò quyết định đến sự thành công của sản phẩm. Một ứng dụng chỉ có code logic thôi là chưa đủ; nó cần có hình ảnh đẹp, âm thanh sống động, font chữ phù hợp và các file dữ liệu cấu hình để hoạt động linh hoạt. Tất cả những file tĩnh này được gọi chung là **Assets**.

Flutter cung cấp một hệ thống quản lý assets mạnh mẽ và linh hoạt, cho phép nhà phát triển đóng gói các tài nguyên này trực tiếp vào trong ứng dụng. Chương này sẽ hướng dẫn chi tiết cách khai báo, sử dụng và tối ưu hóa assets trong Flutter, từ những hình ảnh đơn giản đến các file dữ liệu phức tạp và font chữ tùy chỉnh.

8.2. Khái niệm về Assets

8.2.1. Assets là gì?

Asset là các file tĩnh (static files) được bao gói cùng với ứng dụng khi thực hiện quá trình build (biên dịch). Khác với code nguồn (Dart code) có thể bị biên dịch thành máy ảo hoặc native code, assets được giữ nguyên vẹn và truy xuất thông qua một hệ thống gọi là **Asset Bundle**.

8.2.2. Các loại Assets phổ biến

Trong Flutter, chúng ta có thể quản lý nhiều loại tài nguyên khác nhau:

- **Hình ảnh:** .png, .jpg, .jpeg, .webp, .gif.
- **Âm thanh:** .mp3, .wav, .ogg.
- **Dữ liệu cấu hình:** .json, .xml, .csv, .txt.
- **Font chữ:** .ttf, .otf.

Việc hiểu rõ loại asset mình đang sử dụng giúp chọn đúng phương thức truy xuất (ví dụ: dùng widget Image cho hình ảnh hay dùng rootBundle cho file text).

8.3. Cấu hình Assets trong pubspec.yaml

Để Flutter nhận diện và đóng gói các file tài nguyên vào ứng dụng, chúng ta bắt buộc phải khai báo chúng trong file pubspec.yaml. Đây là file cấu hình trung tâm của mọi dự án Flutter.

8.3.1. Cấu trúc khai báo

Mở file pubspec.yaml nằm ở thư mục gốc của dự án. Tìm section flutter và thêm khóa assets.

```
flutter:
  uses-material-design: true

  assets:
    - images/logo.png
    - images/icons/
    - data/config.json
```

8.3.2. Những lưu ý quan trọng khi chỉnh sửa YAML

- **Thụt đầu dòng (Indentation):** YAML cực kỳ nhạy cảm với khoảng trắng. Chúng ta phải sử dụng khoảng trắng (space) để thụt đầu dòng, **không được dùng phím Tab**. Thông thường, mức thụt chuẩn là 2 khoảng trắng. Sai sót nhỏ ở đây sẽ khiến project không thể build.
- **Đường dẫn (Path):** Đường dẫn được tính từ thư mục gốc của dự án (nơi chứa file pubspec.yaml).
- Thư mục vs File:
  - Nếu khai báo cụ thể file (ví dụ: images/logo.png), chỉ file đó được đưa vào bundle.

- Nếu khai báo thư mục (ví dụ: images/icons/ - lưu ý dấu / ở cuối), Flutter sẽ quét và đưa **tất cả** file nằm trong thư mục đó vào bundle.

- **Hot Restart:** Sau khi chỉnh sửa pubspec.yaml, chúng ta bắt buộc phải thực hiện **Hot Restart** (nút mũi tên vòng tròn màu cam/xanh) hoặc dừng và chạy lại ứng dụng. **Hot Reload** sẽ không áp dụng thay đổi của file cấu hình này.

8.4. Làm việc với Hình ảnh (Image Assets)

Sau khi đã khai báo, cách đơn giản nhất để hiển thị hình ảnh là sử dụng widget Image.asset.

8.4.1. Cú pháp cơ bản

```
Image.asset('images/logo.png')
```

8.4.2. Các thuộc tính tùy chỉnh

Widget Image.asset chấp nhận nhiều tham số để điều chỉnh cách hiển thị:

```
Center(
  child: Image.asset(
    'images/logo.png',
    width: 100,              // Chiều rộng cố định
    height: 100,             // Chiều cao cố định
    fit: BoxFit.cover,       // Cách thức co giãn
ảnh
    color: Colors.red,       // Phủ màu lên ảnh
(tinting)
  ),
)
```

- **fit (BoxFit):** Quyết định cách ảnh lấp đầy khung chứa.
  - BoxFit.cover: Phóng to ảnh để lấp đầy khung, có thể bị cắt bớt.

- BoxFit.contain: Thu nhỏ ảnh để hiển thị toàn bộ trong khung, có thể để lại khoảng trống.
- BoxFit.fill: Kéo giãn ảnh để lấp đầy khung, có thể làm méo ảnh.

8.5. Resolution Awareness (Đa độ phân giải)

8.5.1. Vấn đề mật độ điểm ảnh

Các thiết bị di động hiện nay có mật độ điểm ảnh (pixel density) khác nhau. Một chiếc điện thoại cũ có thể có mật độ 1.0x, trong khi các flagship hiện đại có thể là 2.0x, 3.0x hoặc cao hơn. Nếu chỉ sử dụng một file ảnh duy nhất, ảnh có thể bị vỡ (pixelated) trên màn hình độ phân giải cao hoặc tốn bộ nhớ không cần thiết trên màn hình thấp.

8.5.2. Cơ chế tự động chọn ảnh của Flutter

Flutter hỗ trợ cơ chế **Resolution Awareness** giúp tự động chọn file ảnh phù hợp nhất với thiết bị đang chạy.

Cách tổ chức thư mục: Chúng ta cần tạo các thư mục con đặc biệt bên cạnh file ảnh gốc.

```
images/
  logo.png        (Ảnh gốc - Dùng cho mật độ 1.0x)
  2.0x/
    logo.png      (Dùng cho mật độ 2.0x)
  3.0x/
    logo.png      (Dùng cho mật độ 3.0x)
```

Khai báo trong pubspec.yaml: Chúng ta chỉ cần khai báo đường dẫn đến ảnh gốc:

```
assets:
  - images/logo.png
```

Khi code gọi Image.asset('images/logo.png'), Flutter sẽ tự động kiểm tra mật độ màn hình. Nếu là màn hình 3.0x, nó sẽ tìm và tải file images/3.0x/logo.png.

Lưu ý: *Ngoài cách dùng thư mục, chúng ta cũng có thể dùng hậu tố tên file (ví dụ: logo@2x.png), nhưng cách dùng thư mục con (2.0x/, 3.0x/) được khuyến nghị vì dễ quản lý hơn trong các dự án lớn.*

8.6. Sử dụng AssetBundle cho dữ liệu phi hình ảnh

Không phải asset nào cũng là hình ảnh để hiển thị trực tiếp bằng widget. Đối với các file dữ liệu như JSON, text, âm thanh, chúng ta cần truy xuất thông qua lớp AssetBundle.

8.6.1. Sử dụng rootBundle

Flutter cung cấp đối tượng rootBundle (thuộc package flutter/services.dart) để truy cập vào bundle mặc định của ứng dụng.

8.6.2. Đọc file văn bản (Async)

Việc đọc dữ liệu từ bundle là một thao tác bất đồng bộ (asynchronous), vì vậy chúng ta cần sử dụng async/await.

```
import 'package:flutter/services.dart' show
rootBundle;
Future<String> loadConfig() async {
  // Load nội dung file dưới dạng string
  String data = await
rootBundle.loadString('data/config.json');
  return data;
}
```

8.6.3. Đọc file dưới dạng ByteData

Đối với file nhị phân (như ảnh cần xử lý thủ công, file âm thanh cho package bên thứ 3), chúng ta dùng load:

```
ByteData data = await
rootBundle.load('images/logo.png');
```

8.6.4. Hiển thị dữ liệu bất đồng bộ

Vì việc load asset là bất đồng bộ, chúng ta thường kết hợp với FutureBuilder để hiển thị dữ liệu lên UI mà không làm đơ ứng dụng:

```
FutureBuilder<String>(
  future: loadConfig(),
  builder: (context, snapshot) {
    if (snapshot.hasData) {
      return Text(snapshot.data!);
    } else if (snapshot.hasError) {
      return Text('Lỗi: ${snapshot.error}');
    }
    return CircularProgressIndicator();//Đang tải
  },
)
```

8.7. Tích hợp Font chữ tùy chỉnh (Custom Fonts)

Sử dụng font chữ mặc định của hệ thống đôi khi không đủ để thể hiện thương hiệu. Flutter cho phép chúng ta nhúng font chữ riêng vào ứng dụng.

8.7.1. Các bước thực hiện

- **Chuẩn bị file:** Tải file font (.ttf hoặc .otf) và copy vào thư mục dự án (ví dụ: fonts/).
- Khai báo trong pubspec.yaml:

```
flutter:
  fonts:
    - family: RobotoCustom # Tên sẽ dùng trong
code
      fonts:
        - asset: fonts/Roboto-Regular.ttf
        - asset: fonts/Roboto-Bold.ttf
          weight: 700    # Khai báo độ đậm
        - asset: fonts/Roboto-Italic.ttf
```

```
          style: italic  # Khai báo style nghiêng
```

- Sử dụng trong Widget:

```
Text(
  'Xin chào Flutter',
  style: TextStyle(
    fontFamily: 'RobotoCustom', // Phải trùng với
'family' trong yaml
    fontSize: 20,
  ),
)
```

8.7.2. Lưu ý

- Tên family là tên tự đặt để gọi trong code, không nhất thiết phải trùng với tên file.
- Cần **Hot Restart** sau khi thêm font.
- Đảm bảo file font không bị lỗi (corrupted) và có bản quyền sử dụng phù hợp.

8.8. Sử dụng Assets từ Package bên thứ 3

Khi chúng ta cài đặt các thư viện từ pub.dev (ví dụ: material_design_icons_flutter), các thư viện này cũng có thể chứa assets riêng.

8.8.1. Cơ chế truy xuất

Để truy cập assets nằm trong một package khác, chúng ta cần thêm tiền tố packages/ vào đường dẫn.

Cú pháp:

```
Image.asset('packages/tên_package/đường_dẫn_trong
_package')
```

Ví dụ: Giả sử package icon_package có file ảnh tại assets/icons/star.png trong thư mục của nó. Khi dùng trong project của chúng ta:

```
Image.asset('packages/icon_package/assets/icons/s
tar.png')
```

8.8.2. Lưu ý

- Chúng ta không cần khai báo assets của package bên thứ 3 vào pubspec.yaml của project chính (vì nó đã được khai báo trong package đó rồi).
- Chúng ta chỉ cần biết đường dẫn chính xác mà package đó cung cấp (thường có trong tài liệu README của package).

8.9. Tối ưu hóa và Best Practices

Quản lý assets không đúng cách có thể làm tăng kích thước ứng dụng (.apk/.ipa) một cách đáng kể, dẫn đến thời gian tải lâu và tốn dung lượng người dùng.

8.9.1. Định dạng file

- **Sử dụng WebP:** Đối với hình ảnh, định dạng .webp của Google cung cấp chất lượng tương đương .png nhưng kích thước file nhỏ hơn đáng kể. Flutter hỗ trợ WebP trên cả iOS và Android.
- **Nén ảnh:** Sử dụng các công cụ như TinyPNG để nén ảnh trước khi đưa vào dự án.

8.9.2. Tổ chức thư mục

Hãy duy trì cấu trúc thư mục rõ ràng ngay từ đầu:

```
assets/
  ├── images/
  │   ├── icons/
  │   ├── backgrounds/
  │   └── logos/
  ├── fonts/
  └── data/
```

Việc này giúp dễ dàng bảo trì và tránh xung đột tên file.

8.9.3. Dọn dẹp Cache

Đôi khi Flutter bị lỗi cache assets (ví dụ: đổi ảnh nhưng app vẫn hiện ảnh cũ). Hãy chạy lệnh sau trong terminal để làm sạch:

```
flutter clean
flutter pub get
```

8.9.4. Kiểm tra kích thước

Sử dụng lệnh sau để phân tích kích thước ứng dụng sau khi build:

```
flutter build apk --analyze-size
```

Lệnh này giúp chúng ta xem asset nào đang chiếm nhiều dung lượng nhất.

8.10. Các lỗi thường gặp và cách khắc phục

Lỗi (Error)	Nguyên nhân phổ biến	Cách khắc phục
Unable to load asset	Đường dẫn sai chính tả hoặc chưa khai báo trong pubspec.yaml.	Kiểm tra lại đường dẫn, kiểm tra indentation trong yaml, thực hiện Hot Restart.
YAML syntax error	Sử dụng phím Tab thay vì Space, hoặc thụt đầu dòng sai.	Dùng Space (2 khoảng trắng), kiểm tra kỹ cấu trúc file yaml.
Font không hiển thị	Sai tên fontFamily trong code so với family trong yaml.	Đảm bảo tên trùng khớp chính xác (case-sensitive).
Ảnh bị mờ	Không chuẩn bị ảnh cho màn hình độ phân giải cao (2x, 3x).	Áp dụng cơ chế Resolution Awareness (thư mục 2.0x, 3.0x).

Lỗi (Error)	Nguyên nhân phổ biến	Cách khắc phục
Null check operator used on null	Truy cập dữ liệu asset trước khi nó load xong.	Sử dụng FutureBuilder hoặc kiểm tra trạng thái snapshot.hasData.

8.11. Bài tập thực hành

Đề bài: Xây dựng màn hình "Hồ sơ cá nhân" (Profile Screen).

Yêu cầu kỹ thuật:

- **Thư mục:** Tạo cấu trúc thư mục assets/images, assets/fonts, assets/data.
- Hình ảnh:
  - Thêm ảnh đại diện (avatar).
  - Áp dụng Resolution Awareness (chuẩn bị ít nhất 2 phiên bản độ phân giải).
  - Sử dụng BoxFit.cover để ảnh hiển thị tròn (kết hợp với CircleAvatar hoặc ClipOval).
- Font chữ:
  - Tích hợp một font chữ không phải mặc định (ví dụ: *Montserrat* hoặc *Roboto*) cho phần Tên người dùng.
- Dữ liệu:
  - Tạo file profile.json chứa thông tin: name, email, bio.
  - Sử dụng rootBundle để đọc file JSON này và hiển thị lên màn hình khi app khởi động.
- **Giao diện:** Bố cục đẹp mắt, sử dụng icon từ assets local cho các thông tin liên hệ.

Gợi ý: Sử dụng FutureBuilder để xử lý việc đọc file JSON bất đồng bộ trước khi hiển thị giao diện chính.

8.12. Tổng kết

Quản lý assets là một kỹ năng nền tảng nhưng cực kỳ quan trọng trong phát triển ứng dụng Flutter. Một quy trình quản lý assets tốt sẽ giúp:

- Ứng dụng có giao diện chuyên nghiệp, đồng bộ thương hiệu.
- Hiệu suất tải trang nhanh hơn nhờ tối ưu hóa hình ảnh.
- Code sạch sẽ, dễ bảo trì nhờ cấu trúc thư mục hợp lý.

Chương 9: Form & Gesture

9.1. Giới thiệu

Trong phát triển ứng dụng di động, giao diện người dùng (UI) không chỉ dừng lại ở việc hiển thị thông tin tĩnh. Một ứng dụng thành công cần có khả năng **tương tác hai chiều**: thu thập dữ liệu từ người dùng và phản hồi lại các hành động chạm, vuốt của họ.

Chương này sẽ tập trung vào hai trụ cột chính của tương tác trong Flutter:

- **Form (Biểu mẫu):** Cơ chế để người dùng nhập liệu (văn bản, mật khẩu, lựa chọn) và cách xác thực dữ liệu đó.
- **Gesture (Cử chỉ):** Cách ứng dụng nhận diện và xử lý các thao tác vật lý trên màn hình cảm ứng như chạm, nhấn giữ, kéo thả hay phóng to.

Hiểu rõ hai khái niệm này là nền tảng để xây dựng các ứng dụng có trải nghiệm người dùng (UX) mượt mà và chuyên nghiệp.

9.2. Xử lý Nhập liệu (Input Handling)

9.2.1. Widget TextField cơ bản

TextField là widget cơ bản nhất trong thư viện Material của Flutter, cho phép người dùng nhập văn bản. Mặc dù trông đơn giản, TextField có nhiều thuộc tính quan trọng để điều chỉnh hành vi:

- **decoration**: Sử dụng InputDecoration để trang trí ô nhập liệu (nhãn label, icon gợi ý, border, màu sắc khi có lỗi).
- **obscureText**: Nếu đặt là true, văn bản sẽ bị ẩn (thường dùng cho trường mật khẩu).
- **keyboardType**: Định nghĩa loại bàn phím hiển thị (ví dụ: TextInputType.emailAddress, TextInputType.number, TextInputType.phone).

- **onChanged**: Hàm callback được gọi mỗi khi người dùng nhập thêm hoặc xóa một ký tự.
- **onSubmitted**: Hàm callback được gọi khi người dùng nhấn nút hành động trên bàn phím (như "Done", "Next", "Search").

9.2.2. Quản lý dữ liệu với TextEditingController

Để lấy được nội dung mà người dùng đã nhập từ TextField ra để xử lý (ví dụ: gửi lên server), chúng ta không thể đọc trực tiếp từ widget mà cần một bộ điều khiển trung gian gọi là TextEditingController.

Cách sử dụng:

```
// Khai báo
final _controller = TextEditingController();
// Lấy giá trị
String value = _controller.text;
// Đặt giá trị lập trình
_controller.text = "Xin chào Flutter";
// Lắng nghe sự thay đổi
_controller.addListener(() {
  print("Văn bản mới: ${_controller.text}");
});
```

> **Cảnh báo quan trọng về Vòng đời (Lifecycle):** TextEditingController chiếm dụng tài nguyên bộ nhớ. Nếu không giải phóng nó khi widget bị hủy, ứng dụng sẽ bị rò rỉ bộ nhớ (Memory Leak), dẫn đến treo máy hoặc crash sau một thời gian sử dụng.

Chúng ta bắt buộc phải ghi đè phương thức dispose() trong class State:

```
@override
void dispose() {
  _controller.dispose(); // Giải phóng bộ nhớ
  super.dispose();
}
```

9.2.3. Điều khiển Focus với FocusNode

Trong các form phức tạp, chúng ta thường cần điều khiển việc hiển thị bàn phím hoặc di chuyển con trỏ giữa các ô nhập liệu. FocusNode là công cụ để làm việc này.

- **Mở bàn phím:** FocusScope.of(context).requestFocus(_focusNode).
- **Đóng bàn phím:** FocusScope.of(context).unfocus().
- **Di chuyển focus:** Khi người dùng nhấn nút "Next" trên bàn phím, chúng ta có thể dùng FocusNode của ô tiếp theo để chuyển con trỏ sang đó tự động.

9.2.4. Form và TextFormField

Trong khi TextField hoạt động độc lập, TextFormField được thiết kế để nằm bên trong widget Form. Sự kết hợp này mang lại khả năng **xác thực (validation)** và **quản lý trạng thái tập trung**.

Cấu trúc chuẩn của một Form:

- **Widget Form**: Bao bọc toàn bộ các trường nhập liệu.
- **GlobalKey<FormState>**: Một chìa khóa duy nhất để quản lý trạng thái của toàn bộ Form (dùng để gọi lệnh validate, save, reset).
- **Widget TextFormField**: Các ô nhập liệu con, có thuộc tính validator.

9.2.5. Xác thực dữ liệu (Validation)

Thuộc tính validator trong TextFormField nhận vào một hàm. Hàm này nhận giá trị hiện tại của ô nhập và trả về:

- **String**: Nếu dữ liệu không hợp lệ (chuỗi này sẽ hiển thị làm thông báo lỗi).
- **null**: Nếu dữ liệu hợp lệ.

Ví dụ:

```
validator: (value) {
  if (value == null || value.isEmpty) {
    return 'Vui lòng không để trống';
  }
  if (!value.contains('@')) {
    return 'Email không đúng định dạng';
  }
  return null; // Hợp lệ
}
```

Để kích hoạt quá trình kiểm tra, chúng ta gọi phương thức validate() thông qua GlobalKey:

```
if (_formKey.currentState!.validate()) {
  // Dữ liệu hợp lệ, tiến hành xử lý
}
```

9.2.6. Lưu và Gửi Form (Save & Submit)

Ngoài việc kiểm tra lỗi, Form còn hỗ trợ lưu trữ dữ liệu cuối cùng trước khi gửi đi thông qua thuộc tính onSaved của TextFormField.

- **onSaved**: Được gọi khi chúng ta thực thi _formKey.currentState!.save().
- **Quy trình Submit chuẩn:**
  - Người dùng nhấn nút "Đăng ký/Đăng nhập".
  - Gọi validate(): Nếu có lỗi, dừng lại và hiển thị lỗi.
  - Nếu không có lỗi, gọi save(): Cập nhật dữ liệu vào các biến trung gian.
  - Gửi dữ liệu đã lưu lên server hoặc xử lý logic tiếp theo.

9.3. Xử lý Cử chỉ (Gesture Handling)

9.3.1. Khái niệm về Gesture

Gesture (cử chỉ) là các hành động vật lý mà người dùng thực hiện trên màn hình cảm ứng. Flutter cung cấp hệ thống nhận diện gesture

ở mức thấp (raw) và mức cao (high-level). Các loại gesture phổ biến bao gồm:

- **Tap:** Chạm nhẹ (một lần hoặc nhiều lần).
- **Long Press:** Nhấn giữ trong một khoảng thời gian.
- **Drag:** Kéo hoặc vuốt ngón tay trên màn hình.
- **Scale:** Dùng hai ngón tay để phóng to hoặc thu nhỏ.

9.3.2. Widget GestureDetector

GestureDetector là một widget "vô hình" (không vẽ gì lên màn hình), dùng để bao bọc một widget con nhằm biến widget đó thành một vùng nhận cảm ứng.

Các callback quan trọng:

- onTap, onTapDown, onTapUp, onTapCancel: Xử lý sự kiện chạm.
- onDoubleTap: Xử lý chạm hai lần.
- onLongPress: Xử lý nhấn giữ.
- onPanUpdate: Xử lý khi ngón tay di chuyển (Drag).
- onScaleUpdate: Xử lý khi phóng to/thu nhỏ.

Ví dụ:

```
GestureDetector(
  onTap: () => print('Đã chạm'),
  onLongPress: () => print('Nhấn giữ'),
  child: Container(
    color: Colors.blue,
    child: Text('Chạm vào tôi'),
  ),
)
```

9.3.3. InkResponse & InkWell (Hiệu ứng Material)

Một nhược điểm của GestureDetector là nó không cung cấp hiệu ứng sóng nước (Ripple Effect) khi chạm, vốn là chuẩn thiết kế của Material Design.

Để có hiệu ứng này, Flutter cung cấp:

- **InkWell**: Sử dụng khi widget nền là Material (ví dụ: đặt trong Card, Container có Material, hoặc Scaffold).
- **InkResponse**: Linh hoạt hơn, không yêu cầu Material trực tiếp nhưng cần widget Ink để vẽ hiệu ứng.

Khuyến nghị: *Sử dụng InkWell cho các nút bấm, danh sách (ListTile) để đảm bảo trải nghiệm người dùng đồng bộ với hệ điều hành Android.*

9.3.4. Xử lý Drag và Scale

Khi xử lý các cử chỉ phức tạp như kéo thả hoặc phóng to, chúng ta cần hiểu về dữ liệu chi tiết mà Flutter cung cấp:

- **Drag (onPanUpdate):** Nhận đối tượng DragUpdateDetails. Thuộc tính delta cho biết khoảng cách di chuyển kể từ khung hình trước đó. Chúng ta cần cộng dồn khoảng cách này vào vị trí hiện tại của widget và gọi setState để vẽ lại.
- **Scale (onScaleUpdate):** Nhận đối tượng ScaleUpdateDetails. Thuộc tính scale cho biết hệ số phóng to (ví dụ: 1.5 nghĩa là phóng to 150%).

Lưu ý hiệu suất: *Khi xử lý Drag/Scale, sự kiện được gọi rất nhiều lần (mỗi khung hình). Hãy đảm bảo logic trong setState nhẹ nhàng để tránh gây giật lag (jank).*

9.3.5. Gesture Disambiguation (Arena)

Trong một số trường hợp, nhiều cử chỉ có thể xảy ra cùng lúc hoặc chồng lấn lên nhau (ví dụ: vừa vuốt để scroll danh sách, vừa chạm để chọn item). Flutter sử dụng cơ chế **Gesture Arena** (Đấu trường cử chỉ) để quyết định gesture nào sẽ thắng.

- Các gesture sẽ "thi đấu" dựa trên thời gian giữ, khoảng cách di chuyển.
- Nếu xảy ra xung đột (ví dụ: ListView scroll bị chặn bởi GestureDetector), chúng ta có thể cần cấu hình lại

HitTestBehavior hoặc sử dụng các widget chuyên biệt như Dismissible.

9.4. Khả năng tiếp cận (Accessibility)

Tương tác người dùng không chỉ dành cho người dùng cảm ứng thông thường. Ứng dụng cần hỗ trợ:

- **Trình đọc màn hình (Screen Reader):** Các widget chỉ có icon cần có label để mô tả ý nghĩa.
- **Kích thước vùng chạm (Touch Target):** Theo chuẩn Material Design, vùng chạm tối thiểu nên là **48x48 dp**. Dù icon nhỏ, hãy bao quanh nó bằng một vùng đệm vô hình đủ lớn để người dùng dễ dàng nhấn trúng.
- **Semantics:** Flutter tự động ánh xạ các gesture sang ngữ nghĩa, nhưng hãy kiểm tra lại bằng công cụ Inspector để đảm bảo người dùng khuyết tật có thể sử dụng app.

9.5. Các lỗi thường gặp (Pitfalls)

- **Quên dispose() Controller:** Đây là lỗi phổ biến nhất gây rò rỉ bộ nhớ. Luôn luôn giải phóng TextEditingController, AnimationController, FocusNode trong phương thức dispose.
- **Validator trả về sai:** Hàm validator phải trả về null khi hợp lệ. Trả về chuỗi rỗng "" vẫn được coi là có lỗi.
- **GestureDetector không hoạt động:**
  - Widget con bên trong có kích thước bằng 0.
  - Bị widget khác che khuất (vấn đề thứ tự xếp chồng trong Stack).
  - Bị chặn bởi IgnorePointer hoặc AbsorbPointer.
- **Quá nhiều setState:** Gọi setState trong mỗi khung hình của gesture drag có thể gây nặng máy. Hãy cân nhắc sử dụng Transform trực tiếp hoặc các widget tối ưu hiệu năng.

9.6. Best Practices

- **Tách biệt Logic:** Không viết logic xử lý dữ liệu phức tạp trực tiếp trong callback của Gesture hay Form. Hãy gọi các hàm từ Controller (theo mô hình MVC, Clean Architecture, hoặc Bloc/Provider).
- **Phản hồi người dùng:** Luôn có phản hồi khi tương tác (ví dụ: hiển thị loading khi submit form, hiệu ứng ripple khi nhấn nút).
- **Tái sử dụng:** Nếu form có nhiều trường nhập liệu giống nhau, hãy tạo một widget CustomTextField riêng để quản lý validation và decoration đồng nhất.
- **Keyboard Actions:** Cấu hình textInputAction (Next, Done, Search) để hỗ trợ điều hướng bàn phím mượt mà.

9.7. Bài tập thực hành

Bài tập 1: Form Đăng ký Xây dựng một màn hình Đăng ký tài khoản bao gồm:

- Họ tên (Bắt buộc).
- Email (Bắt buộc, phải có ký tự @).
- Mật khẩu (Bắt buộc, tối thiểu 6 ký tự).
- Xác nhận mật khẩu (Phải trùng với mật khẩu).
- Nút đăng ký: Chỉ hoạt động khi tất cả các trường hợp lệ.

Bài tập 2: Widget Tương tác Cảm xúc Tạo một widget hình trái tim:

- Khi chạm nhẹ (Tap): Đổi màu sang đỏ.
- Khi nhấn giữ (Long Press): Phóng to lên 1.5 lần trong khi đang giữ, và thu nhỏ lại khi thả tay.
- Sử dụng GestureDetector và AnimatedContainer hoặc Transform.scale.

9.8. Tổng kết

Chương 9 đã cung cấp cái nhìn toàn diện về cách Flutter xử lý tương tác người dùng:

- **Form:** Sử dụng Form, TextFormField và GlobalKey để quản lý nhập liệu và xác thực dữ liệu chặt chẽ.
- **Controller:** Quản lý vòng đời của TextEditingController và FocusNode là bắt buộc để tránh lỗi bộ nhớ.
- **Gesture:** GestureDetector linh hoạt cho mọi loại chạm, InkWell chuẩn hóa hiệu ứng Material.
- **Sự kiện:** Hiểu rõ cách lấy dữ liệu từ DragUpdateDetails hay ScaleUpdateDetails để xử lý chuyển động.
- **UX:** Chú trọng vào phản hồi người dùng, khả năng tiếp cận và hiệu suất render.

Nắm vững chương này sẽ giúp chúng ta xây dựng được những ứng dụng không chỉ đẹp mà còn "thông minh" trong cách phản hồi lại hành động của người dùng.

Chương 10: Animation

10.1. Giới thiệu

Trong phát triển ứng dụng di động hiện đại, giao diện người dùng (UI) không chỉ dừng lại ở việc sắp xếp các thành phần tĩnh. Một ứng dụng chất lượng cao cần có sự sống động, mượt mà và phản hồi nhanh chóng với hành động của người dùng. Đó là vai trò của **Animation (Hoạt ảnh)**.

Animation không chỉ đơn thuần là làm cho ứng dụng "đẹp hơn". Nó đóng vai trò quan trọng trong trải nghiệm người dùng (UX) thông qua các chức năng sau:

- **Phản hồi (Feedback):** Xác nhận hành động của người dùng (ví dụ: nút bấm lún xuống khi chạm, thông báo thành công hiện ra).
- **Điều hướng (Guidance):** Dẫn dắt mắt người dùng đến vị trí mới hoặc thông tin quan trọng (ví dụ: chuyển cảnh mượt mà).
- **Trạng thái (Status):** Hiển thị sự thay đổi trạng thái hệ thống (ví dụ: vòng quay loading, thanh tiến trình).
- **Cảm xúc (Delight):** Tạo ấn tượng tốt, khiến ứng dụng trông chuyên nghiệp và thân thiện.

Nguyên tắc vàng khi làm animation là: **Nhanh** (không làm chậm tác vụ chính) và **Có ý nghĩa** (không làm rối mắt người dùng). Flutter cung cấp một hệ thống animation mạnh mẽ, linh hoạt và dễ sử dụng, hỗ trợ đạt mức hiệu suất 60fps hoặc 120fps trên các thiết bị hiện đại.

10.2. Phân loại Animation trong Flutter

Flutter chia cơ chế animation thành hai loại chính dựa trên mức độ kiểm soát của lập trình viên:

10.2.1. Implicit Animation (Animation ẩn)

- **Cơ chế:** Dựa trên sự thay đổi trạng thái (State). Khi chúng ta thay đổi một thuộc tính (ví dụ: chiều rộng từ 100 lên 200), Flutter tự động tính toán và vẽ các khung hình trung gian để chuyển đổi giữa hai trạng thái đó.
- **Đặc điểm:** Dễ sử dụng, code ngắn gọn, không cần quản lý bộ điều khiển (Controller).
- **Khi nào dùng:** Khi cần các hiệu ứng chuyển đổi cơ bản như đổi màu, đổi kích thước, di chuyển vị trí đơn giản.
- **Widget phổ biến:** AnimatedContainer, AnimatedOpacity, AnimatedPadding, AnimatedAlign.

10.2.2. Explicit Animation (Animation tường minh)

- **Cơ chế:** Lập trình viên kiểm soát từng khung hình của quá trình animation. Chúng ta phải định nghĩa rõ thời gian bắt đầu, kết thúc, cách thức thay đổi giá trị và khi nào dừng lại.
- **Đặc điểm:** Phức tạp hơn, cần sử dụng AnimationController, nhưng mang lại khả năng tùy biến cao nhất.
- **Khi nào dùng:** Khi cần dừng, đảo ngược, lặp lại animation, đồng bộ nhiều animation cùng lúc, hoặc tạo các hiệu ứng vật lý phức tạp.
- **Widget phổ biến:** AnimationController, Tween, AnimatedBuilder, TransformationController.

10.3. Implicit Animation

10.3.1. Cơ chế hoạt động

Với Implicit Animation, chúng ta không cần quan tâm đến việc giá trị thay đổi như thế nào theo thời gian. Chúng ta chỉ cần xác định **giá trị đích** và **thời gian thực hiện (duration)**. Flutter sẽ tự động nội suy (interpolate) giá trị hiện tại sang giá trị đích.

10.3.2. Ví dụ với AnimatedContainer

AnimatedContainer là widget linh hoạt nhất trong nhóm này, cho phép animate nhiều thuộc tính cùng lúc (kích thước, màu sắc, border, shadow...).

Code mẫu:

```
// Biến trạng thái
double _width = 100;

// Widget
AnimatedContainer(
  duration: Duration(seconds: 1), // Thời gian
chạy
  curve: Curves.easeInOut,        // Đường cong
tốc độ
  width: _width,
  color: _width == 100 ? Colors.red :
Colors.blue,
)

// Hành động kích hoạt
setState(() {
  _width = 200; // Flutter tự động animate từ 100
-> 200
});
```

Nhận xét: Chỉ cần gọi setState, widget sẽ tự động xử lý phần còn lại. Đây là cách nhanh nhất để thêm hiệu ứng cho ứng dụng.

10.4. Explicit Animation - Cơ chế cốt lõi

Khi Implicit Animation không đáp ứng được yêu cầu, chúng ta cần đến Explicit Animation. Để hiểu rõ cách nó hoạt động, hãy hình dung một quy trình 5 bước (Pipeline):

- **Ticker:** Giống như nhịp tim, nó đập 60 lần mỗi giây (hoặc theo tần số màn hình), báo hiệu cho hệ thống biết khi nào cần vẽ khung hình mới.
- **AnimationController:** Là "động cơ" chính. Nó quản lý thời gian và sinh ra một giá trị số thực thay đổi liên tục từ **0.0 đến 1.0** trong suốt thời gian animation.
- **Animation:** Đối tượng lắng nghe Controller. Nó giữ giá trị hiện tại (0.0 -> 1.0) và thông báo cho widget biết khi có sự thay đổi.
- **Tween:** Viết tắt của "In-Between". Vì Controller chỉ chạy từ 0.0 đến 1.0, nên Tween có nhiệm vụ biến đổi giá trị này thành giá trị thực tế mong muốn (ví dụ: 0.0 -> 100.0 pixels, hoặc Colors.red -> Colors.blue).
- **Widget (Listener/Builder):** Nhận giá trị đã được Tween chuyển đổi và vẽ lên màn hình.

Sơ đồ luồng: Ticker → AnimationController (0.0 -> 1.0) → Tween (0.0 -> 100.0) → Widget

10.4.1. AnimationController

Đây là thành phần quan trọng nhất. Nó yêu cầu một đối tượng vsync (TickerProvider) để đồng bộ với tần số quét màn hình, giúp tiết kiệm pin khi ứng dụng chạy nền.

Khởi tạo và Quản lý vòng đời:

```
late AnimationController _controller;
@override
void initState() {
  super.initState();
  _controller = AnimationController(
    duration: const Duration(seconds: 2),
    vsync: this, // Bắt buộc phải có
  );
}
```

```
@override
void dispose() {
  _controller.dispose(); // BẮT BUỘC: Giải phóng
bộ nhớ để tránh Memory Leak
  super.dispose();
}
```

Cảnh báo: Quên gọi dispose() cho AnimationController là nguyên nhân hàng đầu gây rò rỉ bộ nhớ và crash ứng dụng Flutter.

10.4.2. Tween (Nội suy giá trị)

AnimationController chỉ sinh ra giá trị từ 0.0 đến 1.0. Để sử dụng giá trị này cho kích thước, màu sắc hay vị trí, ta cần Tween.

Ví dụ:

```
// Biến đổi từ 0.0 -> 1.0 thành 0.0 -> 100.0
(double)
final Animation<double> _scaleAnim =
    Tween<double>(begin: 0, end:
100).animate(_controller);

// Biến đổi màu sắc
final Animation<Color?> _colorAnim =
    ColorTween(begin: Colors.red, end:
Colors.blue).animate(_controller);
```

10.4.3. Curves (Đường cong tốc độ)

Mặc định, animation chạy tuyến tính (Linear) - tốc độ không đổi từ đầu đến cuối, tạo cảm giác cứng nhắc và thiếu tự nhiên. Curve giúp định nghĩa tốc độ chạy (chậm lúc đầu, nhanh ở giữa, chậm lúc cuối...).

Các Curve phổ biến:

- Curves.easeInOut: Mượt mà cả lúc bắt đầu và kết thúc (khuyên dùng).
- Curves.bounceOut: Tạo hiệu ứng nảy khi kết thúc.
- Curves.elasticIn: Tạo hiệu ứng lò xo co giãn.

Áp dụng Curve:

```
final curvedAnimation = CurvedAnimation(
  parent: _controller,
  curve: Curves.easeInOut,
);
```

10.4.4. Hiển thị Animation: Listener vs. Builder

Cách chúng ta cập nhật UI khi animation chạy ảnh hưởng lớn đến hiệu suất.

- **Cách 1: AddListener (Kém hiệu năng)** Chúng ta thêm listener vào controller, mỗi khi giá trị thay đổi thì gọi setState().
  - *Nhược điểm:* setState sẽ build lại toàn bộ widget cha, gây giật lag nếu cây widget lớn.
- **Cách 2: AnimatedBuilder / ListenableBuilder (Khuyên dùng)** Widget này chỉ build lại phần subtree cần thay đổi, các phần UI tĩnh khác sẽ không bị ảnh hưởng.

Code mẫu tối ưu:

```
AnimatedBuilder(
  animation: _controller,
  builder: (context, child) {
    // Chỉ vùng này bị vẽ lại
    return Transform.scale(
      scale: _controller.value,
      child: child, // Widget con tĩnh không bị
rebuild
    );
  },
  child: Container(color: Colors.blue),
)
```

10.5. Điều khiển Animation

AnimationController cung cấp nhiều phương thức để điều khiển luồng chạy:

- **.forward():** Chạy từ 0.0 đến 1.0.
- **.reverse():** Chạy ngược từ 1.0 về 0.0.
- **.stop():** Dừng lại tại vị trí hiện tại.
- **.reset():** Đưa giá trị về 0.0.
- **.repeat():** Lặp lại vô hạn.
- **.fling():** Quăng theo quán tính vật lý.

Chúng ta cũng có thể lắng nghe trạng thái hoàn thành để thực hiện hành động tiếp theo:

```
_controller.addStatusListener((status) {
  if (status == AnimationStatus.completed) {
    print("Animation đã hoàn thành!");
    // Có thể chạy reverse() ở đây
  }
});
```

10.6. Các loại Animation đặc biệt

10.6.1. Hero Animation

Đây là hiệu ứng chuyển cảnh đặc trưng của Flutter, tạo cảm giác một widget "bay" từ màn hình này sang màn hình khác.

Yêu cầu:

- Widget ở màn hình nguồn và đích phải được bao bởi widget Hero.
- Thuộc tính tag của hai Hero phải **trùng nhau** (thường dùng ID duy nhất).
- Widget nên cùng loại (ví dụ cùng là Image hoặc Container).

Ví dụ:

```
// Màn hình 1
Hero(tag: 'image-1', child:
Image.network('url...'))

// Màn hình 2
Hero(tag: 'image-1', child:
Image.network('url...'))
```

10.6.2. Staggered Animation (Animation so le)

Dùng để chạy nhiều animation liên tiếp hoặc chồng chéo thời gian (ví dụ: danh sách hiện ra từng item một).

- **Cách làm:** Sử dụng chung một AnimationController.
- **Công cụ:** Sử dụng Interval trong CurvedAnimation để chia khoảng thời gian chạy cho từng widget con (ví dụ: Item 1 chạy từ 0.0-0.3, Item 2 chạy từ 0.3-0.6...).

10.6.3. Physics-based Animation

Thay vì dùng thời gian cố định (Duration), loại animation này sử dụng các mô phỏng vật lý như lực, ma sát, lò xo (SpringSimulation, FrictionSimulation).

- **Ứng dụng:** Kéo để làm mới (Pull to refresh), vuốt để xóa (Swipe to dismiss), các hiệu ứng kéo thả tự nhiên.
- **Widget hỗ trợ:** Draggable, Dismissible, FlingAnimation.

10.7. Hiệu suất và Best Practices

Để đảm bảo ứng dụng chạy mượt mà (60fps/120fps), hãy tuân thủ các nguyên tắc sau:

- **Ưu tiên Implicit Animation:** Nếu có thể, hãy dùng AnimatedContainer... vì code ngắn và Flutter đã tối ưu sẵn.
- **Sử dụng AnimatedBuilder:** Tránh gọi setState trong addListener của Controller. Hãy dùng AnimatedBuilder để cô lập vùng cần vẽ lại.

- **Quản lý bộ nhớ:** Luôn luôn gọi dispose() cho AnimationController, TextEditingController, FocusNode... khi widget bị hủy.
- **Tránh tính toán nặng:** Không thực hiện các phép tính phức tạp inside hàm builder của animation.
- **Sử dụng const:** Khai báo widget const khi có thể để giảm việc rebuild không cần thiết.
- **Thời lượng hợp lý:** Đừng làm animation quá dài (gây cảm giác app chậm) hoặc quá ngắn (gây giật mắt). Thời gian lý tưởng thường từ 200ms - 500ms.

10.8. Các lỗi thường gặp (Pitfalls)

- **Quên vsync:** Khi khởi tạo Controller mà không truyền vsync: this, animation vẫn chạy ngay cả khi app ở nền, gây tốn pin nghiêm trọng.
- **Quên dispose:** Gây rò rỉ bộ nhớ (Memory Leak), dẫn đến crash ứng dụng sau một thời gian sử dụng.
- **Dùng setState sai chỗ:** Gọi setState trong addListener khiến toàn bộ màn hình bị vẽ lại 60 lần/giây, gây giật lag (jank).
- **Tag Hero bị trùng:** Nếu hai Hero widget khác nhau trên cùng một màn hình có cùng tag, ứng dụng sẽ báo lỗi runtime khi chuyển cảnh.
- **Duration quá dài:** Làm người dùng phải chờ đợi, tạo cảm giác ứng dụng phản hồi chậm.

10.9. Bài tập thực hành

Bài tập 1: Nút bấm Animated (Cơ bản)

- Tạo một nút bấm hình tròn.
- Khi nhấn vào, nút tự động đổi màu và phóng to (Scale) sau đó thu nhỏ về cũ.

- **Yêu cầu:** Sử dụng **Implicit Animation** (AnimatedContainer hoặc AnimatedScale).

Bài tập 2: Quả bóng nảy (Nâng cao)

- Tạo một quả bóng di chuyển từ trên xuống dưới và nảy lên liên tục.
- **Yêu cầu:** Sử dụng **Explicit Animation** (AnimationController + Tween + Curves.bounceOut).
- Sử dụng AnimatedBuilder để vẽ vị quả bóng.
- Đảm bảo đã dispose Controller đúng cách.

Bài tập 3: Thư viện ảnh Hero

- Tạo màn hình danh sách sản phẩm (GridView).
- Khi click vào ảnh sản phẩm, chuyển sang màn hình chi tiết.
- **Yêu cầu:** Implement hiệu ứng **Hero Animation** để ảnh bay từ danh sách sang màn hình chi tiết mượt mà.

10.10. Tổng kết

Chương 10 đã cung cấp kiến thức toàn diện về hệ thống Animation trong Flutter:

- **Phân loại:** Hiểu rõ sự khác biệt giữa Implicit (dễ, nhanh) và Explicit (phức tạp, mạnh mẽ).
- **Cơ chế Explicit:** Nắm vững quy trình Ticker → Controller → Animation → Tween → Widget.
- **Công cụ:** Thành thạo AnimationController, Tween, Curve và AnimatedBuilder.
- **Hiệu ứng đặc biệt:** Biết cách sử dụng Hero cho chuyển cảnh và Interval cho animation so le.
- **Tối ưu:** Luôn ưu tiên hiệu suất bằng cách tránh setState không cần thiết và quản lý bộ nhớ chặt chẽ (dispose).

Animation là chìa khóa để biến một ứng dụng chức năng thành một sản phẩm trải nghiệm cao. Việc luyện tập thường xuyên với các bài tập thực hành sẽ giúp chúng ta làm chủ được sức mạnh này của Flutter.

Chương 11: Lưu trữ dữ liệu cục bộ

11.1. Giới thiệu

Trong quá trình phát triển ứng dụng di động, việc lưu trữ dữ liệu cục bộ (Local Data Storage) đóng một vai trò cực kỳ quan trọng. Không phải lúc nào ứng dụng cũng có thể kết nối với máy chủ (server) hoặc Internet ổn định. Hơn nữa, để mang lại trải nghiệm người dùng mượt mà, ứng dụng cần ghi nhớ các trạng thái, cấu hình cá nhân hóa hay dữ liệu đã tải về trước đó.

Lưu trữ cục bộ giúp ứng dụng đạt được các mục tiêu sau:

- **Duy trì trạng thái (State Persistence):** Giữ lại dữ liệu khi ứng dụng bị đóng hoàn toàn hoặc khởi động lại (ví dụ: chế độ sáng/tối, ngôn ngữ, trạng thái đăng nhập).
- **Hoạt động Offline:** Cho phép người dùng truy cập dữ liệu đã lưu mà không cần kết nối mạng.
- **Cache dữ liệu:** Giảm thiểu số lượng gọi API không cần thiết, giúp ứng dụng tải nhanh hơn và tiết kiệm dữ liệu di động.
- **Lưu trữ tệp tin:** Quản lý ảnh, video, tài liệu tải về hoặc file log lỗi.
- **Bảo mật:** Lưu trữ các thông tin nhạy cảm như token xác thực, mật khẩu một cách an toàn.

Flutter cung cấp một hệ sinh thái các gói thư viện (plugins) phong phú để tương tác với bộ nhớ của thiết bị Android và iOS. Trong chương này, chúng ta sẽ tìm hiểu chi tiết về các phương thức lưu trữ phổ biến nhất: **Shared Preferences**, **SQLite**, **File System** và **Secure Storage**.

11.2. Tổng quan các phương thức lưu trữ

Trước khi đi vào chi tiết kỹ thuật, lập trình viên cần hiểu rõ đặc điểm của từng phương thức để lựa chọn công cụ phù hợp cho từng bài toán cụ thể.

- **Shared Preferences:** Lưu trữ dữ liệu đơn giản dưới dạng cặp khóa-giá trị (Key-Value). Phù hợp cho các cấu hình nhỏ, dữ liệu nguyên thủy (int, bool, string).
- **SQLite (qua sqflite):** Cơ sở dữ liệu quan hệ (Relational Database). Phù hợp cho dữ liệu có cấu trúc phức tạp, nhiều bảng, cần truy vấn lọc và sắp xếp.
- **File System:** Lưu trữ dưới dạng tệp tin vật lý. Phù hợp cho ảnh, video, file âm thanh hoặc dữ liệu nhị phân lớn.
- **Secure Storage:** Lưu trữ dữ liệu mã hóa trong vùng bảo mật của hệ điều hành (Keychain trên iOS, Keystore trên Android). Chỉ dùng cho dữ liệu nhạy cảm.
- **NoSQL Local DB (Hive/Isar):** Các giải pháp hiện đại, hiệu năng cao, không cần cấu trúc SQL chặt chẽ (sẽ đề cập ngắn gọn ở phần so sánh).

11.3. Shared Preferences

11.3.1. Khái niệm

Shared Preferences là cơ chế lưu trữ nhẹ nhất trong Flutter. Nó hoạt động dựa trên nguyên lý Key-Value, tương tự như một bảng từ điển đơn giản. Dữ liệu được lưu vĩnh viễn cho đến khi người dùng xóa ứng dụng hoặc gọi lệnh xóa rõ ràng.

Dữ liệu hỗ trợ: int, double, bool, String, List<String>.

Hạn chế: Không lưu được object phức tạp (trừ khi chuyển thành JSON string), không phù hợp cho dữ liệu lớn vì hiệu năng giảm khi dữ liệu phình to.

11.3.2. Cài đặt và sử dụng

Để sử dụng, chúng ta cần thêm gói thư viện vào file pubspec.yaml:

```
dependencies:
  shared_preferences: ^2.2.0
```

Ví dụ minh họa: Dưới đây là cách khởi tạo, lưu, đọc và xóa dữ liệu bằng Shared Preferences. Lưu ý rằng tất cả các thao tác đều là bất đồng bộ (async), do đó cần sử dụng từ khóa await.

```
import
'package:shared_preferences/shared_preferences.da
rt';
class PreferenceService {
  // Lấy instance của SharedPreferences
  Future<SharedPreferences> get _prefs async =>
      await SharedPreferences.getInstance();

  // Lưu dữ liệu
  Future<void> saveUserSettings({required bool
isDarkMode, required String username}) async {
    final prefs = await _prefs;
    await prefs.setBool('is_dark_mode',
isDarkMode);
    await prefs.setString('username', username);
  }

  // Đọc dữ liệu
  Future<Map<String, dynamic>> getUserSettings()
async {
    final prefs = await _prefs;
    // Sử dụng toán tử ?? để gán giá trị mặc định
nếu key không tồn tại
    return {
      'is_dark_mode':
prefs.getBool('is_dark_mode') ?? false,
      'username': prefs.getString('username') ??
'Guest',
    };
  }

  // Xóa dữ liệu cụ thể
  Future<void> logout() async {
```

```
    final prefs = await _prefs;
    await prefs.remove('username'); // Xóa từng
key
    // Hoặc await prefs.clear(); để xóa toàn bộ
  }
}
```

Lưu ý quan trọng: *Shared Preferences không đồng bộ dữ liệu giữa các tiến trình (process) ngay lập tức. Nó phù hợp nhất cho dữ liệu cấu hình cá nhân của người dùng trong một phiên làm việc.*

11.4. Cơ sở dữ liệu SQLite với sqflite

11.4.1. Khái niệm

Khi ứng dụng cần quản lý lượng dữ liệu lớn có cấu trúc (ví dụ: danh sách sản phẩm, đơn hàng, danh bạ), Shared Preferences sẽ không đáp ứng được. Lúc này, **SQLite** là giải pháp tiêu chuẩn. SQLite là cơ sở dữ liệu quan hệ nhúng (embedded), nghĩa là nó chạy trực tiếp trên thiết bị mà không cần máy chủ database riêng biệt.

Trong Flutter, gói sqflite cung cấp wrapper để tương tác với SQLite native trên Android và iOS.

11.4.2. Cài đặt

```
dependencies:
  sqflite: ^2.3.0
  path: ^1.8.0 # Gói hỗ trợ xử lý đường dẫn file
```

11.4.3. Mô hình Database Helper

Để quản lý kết nối database hiệu quả, chúng ta thường sử dụng mô hình **Singleton**. Điều này đảm bảo chỉ có một instance của database được mở trong suốt vòng đời ứng dụng, tránh xung đột tài nguyên.

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
```

```
class DatabaseHelper {
  static final DatabaseHelper instance =
DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('app_database.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async
{
    // Lấy đường dẫn thư mục database mặc định
của hệ điều hành
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(
      path,
      version: 1,
      onCreate: _createDB,
    );
  }

  // Hàm tạo bảng khi database mới được khởi tạo
  Future<void> _createDB(Database db, int
version) async {
    await db.execute('''
      CREATE TABLE users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER,
        email TEXT UNIQUE
```

```
        )
      ''');
    }
  }
```

11.4.4. Thao tác CRUD (Create, Read, Update, Delete)

Sau khi có instance của Database, chúng ta có thể thực hiện các truy vấn SQL chuẩn.

- Thêm dữ liệu (Insert):

```
final db = await
DatabaseHelper.instance.database;
await db.insert('users', {'name': 'Nguyen Van A',
'age': 25, 'email': 'a@example.com'});
```

- Đọc dữ liệu (Query):

```
// Lấy tất cả người dùng
final List<Map<String, dynamic>> maps = await
db.query('users');

// Lấy theo điều kiện (WHERE)
final result = await db.query(
  'users',
  where: 'age > ?',
  whereArgs: [18], // Sử dụng whereArgs để tránh
SQL Injection
  orderBy: 'name ASC'
);
```

- Cập nhật (Update) và Xóa (Delete):

```
await db.update(
  'users',
  {'age': 26},
  where: 'id = ?',
  whereArgs: [1]
);
```

```
await db.delete('users', where: 'id = ?',
whereArgs: [1]);
```

Chuyển đổi dữ liệu: Kết quả trả về từ query là `List<Map<String, dynamic>>`. Trong thực tế, chúng ta nên tạo các lớp Model (ví dụ: class User) và viết phương thức fromMap để chuyển đổi dữ liệu thô thành object dễ sử dụng.

11.5. Lưu trữ tệp tin (File System)

11.5.1. Quản lý đường dẫn với path_provider

Mỗi hệ điều hành có quy định riêng về nơi ứng dụng được phép lưu file. Gói path_provider giúp chúng ta lấy các đường dẫn an toàn này mà không cần hard-code path.

Các thư mục quan trọng:

- getApplicationDocumentsDirectory(): Thư mục tài liệu. Dữ liệu ở đây không bị xóa trừ khi người dùng gỡ ứng dụng. Dùng cho file quan trọng.
- getTemporaryDirectory(): Thư mục tạm (Cache). Hệ thống có thể xóa bất cứ lúc nào khi cần giải phóng bộ nhớ.
- getExternalStorageDirectory() (Android): Thư mục chung (cần xin permission).

11.5.2. Thao tác với file

Sử dụng thư viện dart:io để đọc/ghi file.

```
import 'dart:io';
import
'package:path_provider/path_provider.dart';

Future<void> saveNoteToFile(String content) async
{
```

```
  final directory = await
getApplicationDocumentsDirectory();
  final file =
File('${directory.path}/note.txt');

  try {
    // Ghi dữ liệu vào file
    await file.writeAsString(content);

    // Đọc dữ liệu từ file
    String savedContent = await
file.readAsString();
    print('Nội dung đã lưu: $savedContent');
  } catch (e) {
    print('Lỗi khi thao tác file: $e');
  }
}
```

Lưu ý về Permission: Trên Android 10 trở lên, việc truy cập bộ nhớ chung bị hạn chế (Scoped Storage). Nếu chỉ lưu trong thư mục riêng của ứng dụng (getApplicationDocumentsDirectory), chúng ta thường không cần xin permission phức tạp.

11.6. Bảo mật dữ liệu với Secure Storage

11.6.1. Vấn đề bảo mật

Dữ liệu lưu trong Shared Preferences hay SQLite thực chất là các file văn bản thuần (plain text) nằm trong bộ nhớ máy. Nếu thiết bị bị root (Android) hoặc jailbreak (iOS), hacker có thể dễ dàng đọc được các file này. Do đó, **không bao giờ** lưu mật khẩu hay access token nhạy cảm ở hai nơi này.

11.6.2. Giải pháp Flutter Secure Storage

Gói flutter_secure_storage mã hóa dữ liệu trước khi lưu vào vùng bảo mật của hệ điều hành:

- **iOS:** Sử dụng Keychain.
- **Android:** Sử dụng Keystore.

Cài đặt:

```
dependencies:
  flutter_secure_storage: ^9.0.0
```

Sử dụng:

```
import
'package:flutter_secure_storage/flutter_secure_st
orage.dart';

final storage = FlutterSecureStorage();

// Lưu token
await storage.write(key: 'auth_token', value:
'eyJhbGciOiJIUzI1NiIs...');

// Đọc token
String? token = await storage.read(key:
'auth_token');

// Xóa token
await storage.delete(key: 'auth_token');
```

Lưu ý: *Thao tác với Secure Storage chậm hơn Shared Preferences, chỉ nên dùng cho dữ liệu thực sự nhạy cảm.*

11.7. So sánh và lựa chọn phương thức

Bảng dưới đây tóm tắt giúp chúng ta ra quyết định khi thiết kế ứng dụng:

Phương thức	**Dữ liệu phù hợp**	**Độ phức tạp**	**Bảo mật**	**Hiệu năng**
Shared Preferences	Cấu hình, trạng thái UI (bool, string nhỏ)	Thấp	Thấp	Rất cao
SQLite (sqflite)	Dữ liệu quan hệ, danh sách lớn, cần query	Trung bình	Thấp	Cao
File System	Ảnh, video, file log, dữ liệu binary	Trung bình	Thấp	Tùy kích thước file
Secure Storage	Token, mật khẩu, PIN	Thấp	**Rất cao**	Trung bình
Hive/Isar	Dữ liệu NoSQL, hiệu năng cực cao	Cao	Thấp	**Rất cao**

11.8. Best Practices

Để xây dựng ứng dụng bền vững, chúng ta nên tuân thủ các nguyên tắc sau:

- **Luôn sử dụng Async/Await:** Thao tác I/O (đọc/ghi disk) luôn tốn thời gian. Hãy thực hiện chúng bất đồng bộ để không làm đơ giao diện người dùng (Main Thread).
- **Mô hình Repository:** Không gọi trực tiếp database từ Widget. Hãy tạo một lớp Repository đứng giữa. Widget gọi Repository, Repository quyết định lấy dữ liệu từ DB hay API. Điều này giúp code dễ kiểm thử (unit test) và dễ bảo trì.

- **Xử lý ngoại lệ (Error Handling):** Luôn bao quanh các thao tác lưu trữ bằng khối try-catch. Disk có thể đầy, permission có thể bị từ chối, file có thể bị hỏng.
- **Database Migration:** Khi cập nhật ứng dụng và thay đổi cấu trúc bảng (thêm cột, đổi tên bảng), chúng ta cần tăng version trong openDatabase và viết logic trong onUpgrade để chuyển đổi dữ liệu cũ sang mới, tránh lỗi crash khi người dùng cập nhật app.
- **Dọn dẹp dữ liệu:** Đối với file tạm hoặc cache ảnh, hãy có cơ chế xóa cũ khi dung lượng vượt quá mức cho phép để tránh làm đầy bộ nhớ thiết bị.

11.9. Bài tập thực hành

Đề bài: Xây dựng ứng dụng "Quản lý chi tiêu cá nhân" (Expense Tracker) với các yêu cầu sau:

- **Lưu trữ cấu hình:** Sử dụng Shared Preferences để lưu loại tiền tệ mặc định (VND, USD) và chế độ giao diện (Sáng/Tối).
- **Lưu trữ giao dịch:** Sử dụng SQLite để lưu danh sách các khoản chi (Ngày, Số tiền, Danh mục, Ghi chú). Hỗ trợ thêm, sửa, xóa và lọc theo tháng.
- **Lưu trữ đăng nhập:** Giả lập chức năng đăng nhập, lưu auth_token vào Flutter Secure Storage.
- **Xuất dữ liệu:** Tính năng xuất báo cáo chi tiêu ra file .csv hoặc .txt lưu vào thư mục tài liệu của ứng dụng.

Gợi ý: Hãy áp dụng mô hình Repository để tách biệt logic xử lý dữ liệu khỏi giao diện người dùng.

11.10. Tổng kết

Chương 11 đã cung cấp cái nhìn toàn diện về cách lưu trữ dữ liệu trong Flutter. Từ những dữ liệu đơn giản nhất với Shared Preferences đến các hệ thống phức tạp với SQLite, hay yêu cầu bảo mật cao với

Secure Storage. Việc lựa chọn đúng công cụ lưu trữ là yếu tố then chốt để xây dựng ứng dụng hoạt động ổn định, nhanh chóng và an toàn.

Chương 12: Packages và Plugin

12.1. Giới thiệu

Trong quá trình phát triển ứng dụng di động, việc viết mọi thứ từ đầu (from scratch) là không khả thi và kém hiệu quả. Flutter sở hữu một hệ sinh thái phong phú cho phép các nhà phát triển tái sử dụng mã nguồn, tích hợp các tính năng phức tạp và truy cập vào phần cứng thiết bị thông qua **Packages** và **Plugins**.

Chương này sẽ cung cấp cho chúng ta kiến thức nền tảng về cách tìm kiếm, đánh giá, cài đặt và sử dụng các thư viện bên thứ ba trong Flutter. Đồng thời, chúng ta cũng sẽ tìm hiểu sâu hơn về cơ chế hoạt động của Plugin và cách xử lý các vấn đề thường gặp khi tích hợp chúng vào dự án.

12.2. Tại sao cần sử dụng Packages & Plugins?

Trong phát triển phần mềm hiện đại, nguyên tắc **"Don't Reinvent The Wheel"** (Đừng phát minh lại bánh xe) luôn được ưu tiên. Việc sử dụng các thư viện có sẵn mang lại những lợi ích sau:

- **Tiết kiệm thời gian và công sức:** Thay vì viết lại các chức năng phức tạp như gửi yêu cầu HTTP, kết nối cơ sở dữ liệu, hay xử lý bản đồ, chúng ta có thể sử dụng các thư viện đã được kiểm chứng.
- **Tận dụng sức mạnh cộng đồng:** Hàng ngàn thư viện miễn phí, mã nguồn mở được bảo trì và cập nhật thường xuyên bởi cộng đồng lập trình viên toàn cầu.
- **Truy cập tính năng Native:** Flutter là một framework đa nền tảng, nhưng đôi khi chúng ta cần truy cập vào các API đặc thù của hệ điều hành (như cảm biến vân tay, Bluetooth, GPS). Plugin đóng vai trò là cầu nối để thực hiện điều này.

12.3. Phân biệt Package và Plugin

Mặc dù thường được gọi chung là "Package" trong cộng đồng, nhưng về mặt kỹ thuật, có sự khác biệt rõ ràng giữa **Package** và **Plugin**.

12.3.1. Package (Gói thư viện thuần)

- **Định nghĩa:** Là một thư viện chứa mã nguồn **Dart thuần**.
- **Chức năng:** Cung cấp các tiện ích, widget giao diện, logic xử lý dữ liệu, helper functions...
- **Đặc điểm:** Không chứa code nền tảng (Native code). Chạy được trên mọi nền tảng mà Flutter hỗ trợ (Mobile, Web, Desktop) mà không cần cấu hình thêm về phía hệ điều hành.
- **Ví dụ:**
  - http: Gửi request HTTP.
  - provider: Quản lý trạng thái (State Management).
  - google_fonts: Sử dụng font chữ Google.

12.3.2. Plugin (Gói cắm mở rộng)

- **Định nghĩa:** Là một loại Package đặc biệt, chứa mã nguồn **Dart** và mã nguồn **Native** (Java/Kotlin cho Android, Objective-C/Swift cho iOS).
- **Chức năng:** Truy cập vào **API của hệ điều hành** hoặc **Phần cứng**.
- **Đặc điểm:** Cần cấu hình thêm trên Android (AndroidManifest.xml) và iOS (Info.plist). Hoạt động như một cầu nối (Bridge) giữa Flutter và Native.
- **Ví dụ:**
  - camera: Truy cập camera thiết bị.
  - geolocator: Lấy vị trí GPS.
  - shared_preferences: Lưu dữ liệu cục bộ.

12.3.3. Bảng so sánh tóm tắt

Đặc điểm	**Package**	**Plugin**
Ngôn ngữ	100% Dart	Dart + Native (Java, Kotlin, Swift, Obj-C)
Truy cập phần cứng	Không	Có
Cấu hình nền tảng	Không cần (thường vậy)	Cần (Permission, Manifest...)
Mục đích	Logic, UI, Tiện ích	Tính năng hệ thống, Phần cứng
Ví dụ	dio, flutter_bloc	image_picker, flutter_local_notifications

12.4. Kho lưu trữ pub.dev

pub.dev là kho lưu trữ chính thức các package của Dart và Flutter. Đây là nơi đầu tiên chúng ta nên tìm kiếm khi cần thêm tính năng vào ứng dụng.

- **Địa chỉ:** https://pub.dev
- **Cách tìm kiếm:** Gõ từ khóa chức năng (ví dụ: "http", "camera") vào thanh tìm kiếm.
- **Đánh giá chất lượng thư viện:** Khi chọn một package, hãy xem xét các chỉ số sau để đảm bảo độ tin cậy:
  - **Likes:** Số lượng người thích (càng cao càng tốt).
  - **Popularity:** Độ phổ biến trong cộng đồng.
  - **Pub Points:** Điểm chất lượng code (tài liệu, phân tích tĩnh...).

- **Last updated:** Thời gian cập nhật gần nhất. Lưu ý: Tránh sử dụng các thư viện đã quá 1-2 năm không cập nhật.
- **Flutter Favorite:** Nhãn hiệu đặc biệt dành cho các package chất lượng cao được đội ngũ Flutter xác nhận.

12.5. Cài đặt và Quản lý phiên bản

12.5.1. Cách cài đặt

Có hai cách chính để thêm một package vào dự án Flutter:

Cách 1: Sử dụng câu lệnh (Khuyên dùng) Mở terminal tại thư mục dự án và chạy:

```
flutter pub add ten_package
# Ví dụ:
flutter pub add http
```

Cách này sẽ tự động thêm tên package và phiên bản phù hợp vào file pubspec.yaml và chạy lệnh flutter pub get.

Cách 2: Sửa file pubspec.yaml thủ công Thêm tên package vào phần dependencies:

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.1.0       # Tên package: ^Phiên bản
  camera: ^0.10.5
```

Sau đó chạy lệnh:

```
flutter pub get
```

12.5.2. Quản lý phiên bản (Versioning)

Hiểu về cú pháp phiên bản là rất quan trọng để tránh lỗi xung đột sau này:

- ^1.2.3 (Caret syntax): Chấp nhận cập nhật từ phiên bản 1.2.3 lên các phiên bản mới hơn nhưng **nhỏ hơn 2.0.0**. Đây là cách an

toàn nhất để nhận các bản vá lỗi mà không bị thay đổi lớn (breaking changes).

- 1.2.3: Khóa cứng vào chính xác phiên bản này.
- any: Chấp nhận mọi phiên bản (Không khuyên dùng vì dễ gây lỗi).

12.6. Sử dụng Package trong Code

Sau khi cài đặt, quy trình sử dụng thường bao gồm các bước sau:

- **Import:** Khai báo thư viện ở đầu file Dart.

```
import 'package:http/http.dart' as http;
import 'package:camera/camera.dart';
```

- **Khởi tạo (Nếu cần):** Một số plugin yêu cầu khởi tạo trước khi dùng (thường là trong hàm main()).

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(); // Ví dụ với
Firebase
  runApp(MyApp());
}
```

- **Sử dụng:** Gọi các hàm, class được cung cấp bởi package.

```
final response = await
http.get(Uri.parse('https://api.example.com'));
if (response.statusCode == 200) {
  // Xử lý dữ liệu
}
```

- **Đọc tài liệu:** Luôn đọc tab **Readme** và **Example** trên pub.dev để biết cách sử dụng chính xác nhất.

12.7. Cơ chế hoạt động của Plugin (Platform Channels)

Một câu hỏi thường gặp là: *"Làm thế nào mà code Dart có thể gọi được camera của Android/iOS?"*. Câu trả lời nằm ở **Platform Channels**.

Vì Dart không thể trực tiếp gọi API của Android hay iOS, Flutter sử dụng cơ chế truyền tin nhắn bất đồng bộ để giao tiếp:

- **Flutter Client (Dart):** Gửi yêu cầu thông qua một **Channel** (thường là MethodChannel).
- **Channel:** Đóng vai trò là đường truyền tin nhắn nhị phân.
- **Native Host (Android/iOS):** Lắng nghe trên Channel đó. Khi nhận được yêu cầu, code Native (Java/Kotlin/Swift) sẽ thực thi chức năng phần cứng và gửi kết quả ngược lại cho Flutter.

Quy trình cơ bản:

- Dart gọi `invokeMethod('tên_hàm', dữ_liệu)`.
- Native nhận `onMethodCall`, kiểm tra tên hàm và thực thi.
- Native trả kết quả về qua `result.success()` hoặc `result.error()`.

Lưu ý: *Khi sử dụng Plugin có sẵn, chúng ta không cần viết code Platform Channels này vì nó đã được đóng gói sẵn trong thư viện.*

12.8. Các vấn đề thường gặp và lưu ý

Khi tích hợp Package/Plugin, sinh viên và lập trình viên thường gặp các lỗi sau:

12.8.1. Quyền truy cập (Permissions)

Đây là lỗi phổ biến nhất với Plugin. Các tính năng liên quan đến phần cứng (Camera, Location, Storage...) đều cần được cấp quyền.

- **Android:** Phải khai báo trong file android/app/src/main/AndroidManifest.xml.

```
<uses-permission
android:name="android.permission.CAMERA"/>
```

- **iOS:** Phải khai báo trong file ios/Runner/Info.plist kèm theo mô tả lý do sử dụng.

```
<key>NSCameraUsageDescription</key>
<string>Ứng dụng cần truy cập camera để chụp
ảnh</string>
```

Nếu quên bước này, ứng dụng sẽ bị crash ngay khi gọi tính năng hoặc bị từ chối khi đưa lên Store.

12.8.2. Xung đột phiên bản (Dependency Conflicts)

Xảy ra khi hai package khác nhau trong dự án cùng yêu cầu một thư viện con nhưng ở hai phiên bản khác nhau.

Khắc phục: Kiểm tra thông báo lỗi trong terminal. Có thể sử dụng dependency_overrides trong pubspec.yaml để ép buộc dùng một phiên bản chung (cần cẩn thận), hoặc tìm package thay thế tương thích hơn.

12.8.3. Null Safety

Kể từ Dart 2.12, **Null Safety** là tiêu chuẩn. Hãy ưu tiên chọn các package đã hỗ trợ Null Safety (thường có phiên bản >= 1.0.0 hoặc ghi rõ trong mô tả). Sử dụng package cũ không hỗ trợ Null Safety có thể gây lỗi compile.

12.8.4. Cấu hình Build

Một số plugin yêu cầu cấu hình thêm trong build.gradle (Android) hoặc Podfile (iOS), ví dụ như yêu cầu phiên bản Android tối thiểu (minSdkVersion) hoặc phiên bản iOS tối thiểu. Hãy đọc kỹ mục **Installation** trên pub.dev.

12.9. Tạo Package/Plugin tuỳ biến

Chúng ta có thể tạo ra thư viện riêng để sử dụng cho nhiều dự án hoặc chia sẻ cho cộng đồng.

- Khi nào cần tạo?
  - Muốn modular hóa dự án lớn.
  - Muốn tái sử dụng một nhóm các widget hoặc logic chung.
  - Muốn đóng góp mã nguồn mở.
- Câu lệnh tạo:
  - Tạo Package (Dart thuần):

```
flutter create --template=package ten_package
```

  - Tạo Plugin (Có Native code):

```
flutter create --template=plugin ten_plugin
```

- **Cấu trúc:** Thư mục lib/ chứa code Dart, android/ và ios/ chứa code native (đối với plugin). Thư mục example/ chứa một ứng dụng mẫu để kiểm tra hoạt động của package.

12.10. Best Practices

Để đảm bảo dự án ổn định và dễ bảo trì, hãy tuân thủ các nguyên tắc sau:

- **Kiểm tra độ tin cậy:** Xem số lượt tải, ngày cập nhật cuối, số issue trên GitHub của package trước khi dùng.
- **Tránh lạm dụng:** Không cài quá nhiều package cho cùng một chức năng. Hãy chọn package tốt nhất và gắn bó với nó.
- **Quản lý phiên bản chặt chẽ:** Luôn khóa phiên bản (lock version) khi đưa vào production để tránh việc tự động cập nhật lên phiên bản mới có thể gây lỗi (breaking changes).

- **Đọc Changelog:** Trước khi cập nhật version của package, hãy đọc file thay đổi để xem có ảnh hưởng gì đến code hiện tại không.
- **Bảo mật:** Không commit các file cấu hình chứa thông tin nhạy cảm (như key.properties hay file cấu hình API key) lên Git public.

12.11. Bài tập thực hành

- **Bài tập 1:** Tạo một dự án Flutter mới. Tìm và tích hợp package google_fonts để thay đổi font chữ cho toàn bộ ứng dụng.
- **Bài tập 2:** Tích hợp package image_picker để cho phép người dùng chọn ảnh từ thư viện hoặc chụp ảnh mới.
  - *Yêu cầu:* Cấu hình đầy đủ Permission cho cả Android và iOS.
  - *Yêu cầu:* Hiển thị ảnh đã chọn lên màn hình.
- **Bài tập 3:** Viết một báo cáo ngắn (1 trang) về một package chúng ta tâm đắc nhất, nêu rõ công dụng, cách cài đặt và lý do tại sao nên sử dụng nó.

12.12. Tổng kết

Packages và Plugins là trái tim của hệ sinh thái Flutter, giúp tăng tốc độ phát triển ứng dụng lên gấp nhiều lần.

- **Package** dùng cho logic Dart thuần.
- **Plugin** dùng để truy cập tính năng phần cứng/hệ thống.
- Việc tìm kiếm trên **pub.dev**, cài đặt qua pubspec.yaml và cấu hình **Permissions** là các kỹ năng bắt buộc phải thành thạo.
- Hiểu cơ chế **Platform Channels** giúp chúng ta có cái nhìn sâu sắc hơn về cách Flutter hoạt động dưới nền tảng.